Non-Convex Optimization in Neural Networks: A Survey of Gradient Dynamics from Saddle Escape to the Edge of Stability

Faris Nasser and Salah Moussa

***Abstract*—Training a deep neural network means minimizing a high-dimensional, non-convex loss function, a task that is NP-hard in the worst case. Yet plain gradient descent and stochastic gradient descent reliably reach low training loss in practice. This survey examines one thread of the effort to explain this gap: what unmodified gradient methods do when the gradient-descent map is locally unstable. Near a critical point, gradient descent multiplies each curvature direction by $1 - \eta\lambda$, where η is the step size and λ a Hessian eigenvalue. This multiplier exceeds one in magnitude in two cases. Negative curvature ($\lambda < 0$) is the mechanism by which gradient descent leaves saddle points. Excess curvature ($\lambda > 2/\eta$) is the mechanism behind catapults and the edge of stability. We organize the literature around two research questions: when and how fast unmodified gradient methods escape saddles, and what happens when the step size exceeds the classical stability limit. For the first, we review asymptotic avoidance theorems, exponential worst-case escape times, noise-based escape guarantees, and saddle-to-saddle learning. For the second, we trace the large-step literature from dynamical stability and the break-even point, through catapults and the edge of stability, to self-stabilization, central flows, and the adaptive and stochastic edges of stability. Throughout, we distinguish between what is proved and what is observed. We provide comparative tables of guarantees, stability thresholds, and assumptions, and a set of original simulations that make the central phenomena concrete. We conclude with open problems, including degenerate saddles, a theory of stochastic and adaptive large-step training, and the dual role of instability as both an obstacle and a driver of learning.**

***Index Terms*—Non-convex optimization, saddle points, gradient descent, stochastic gradient descent, linear stability, sharpness, edge of stability, catapult dynamics, saddle-to-saddle dynamics, deep learning theory.**

I. Introduction

DEEP neural networks (DNNs) are trained by minimizing an empirical loss over millions or billions of parameters [1]. Because the network output is a composition of linear maps and non-linear activations, this loss is non-convex in the parameters. Classical optimization theory treats such problems with considerable caution. Deciding whether a point is even a local minimum of a general smooth function is NP-hard [2]: it belongs to a class of problems for which no known algorithm solves every instance in a time that grows only polynomially with the size of the problem. Likewise, training a network with only three nodes, two hidden units and one output unit, is NP-complete in the worst case [3]. Nevertheless, plain gradient descent (GD), which repeatedly steps against the gradient, and its stochastic variant (SGD), which computes each gradient on a small random subset of the data, routinely drive the training loss of modern networks close to zero. Explaining this apparent contradiction has become one of the central problems of deep learning theory.

The explanation that has emerged over the past decade is not a single theorem but a layered story. Early work asked whether neural-network losses have bad local minima, and found that, for linear networks, every local minimum is global [4]. In 2014, Dauphin et al. [5] shifted the focus by arguing that in high dimensions, saddle points (flat points that curve up in some directions and down in others, Section II-C) surrounded by long plateaus, rather than poor local minima, are the dominant obstacle. This motivated a rigorous theory of strict saddles [6]: if every saddle has a direction along which the loss curves strictly downward, then gradient methods avoid them from almost every starting point [7] and, with suitable randomness, escape them efficiently [8]. In parallel, the overparameterization literature, on networks with far more parameters than needed to fit the data, showed that sufficiently wide networks can be trained to global optima from random initialization, often without reasoning about saddles at all [9]–[11]. Most recently, attention has turned to the dynamics actually observed in practice, where large step sizes push training to the edge of stability [12], a regime in which the sharpest curvature of the loss hovers at the largest value that the step size can tolerate. There, loss curves exhibit catapults, sudden spikes after which training settles in a flatter region [13], and networks pass through a sequence of saddles while learning progressively more complex functions [14].

This survey follows one thread through this literature. Near a critical point, GD multiplies the component of the error along each Hessian eigenvector by $1 - \eta\lambda$, where η is the step size and λ the corresponding eigenvalue (Section II-F). The iterates are pushed away from the critical point whenever $|1-\eta\lambda| > 1$, and this happens in exactly two cases. If $\lambda < 0$, the direction has negative curvature, and this is the mechanism by which GD leaves strict saddles. If $\lambda > 2/\eta$, the curvature is too large for the step size, and this is the mechanism behind catapults and the edge of stability. The two cases have been studied by largely separate communities, with different questions and different tools. The saddle literature asks whether and how fast GD escapes. The large-step literature asks why training still converges when it is unstable. Yet both describe the same

F. Nasser and S. Moussa are with the College of Engineering, American University of Sharjah, Sharjah, United Arab Emirates. This survey was prepared for the course MLR 570 under the supervision of Dr. Mohamad Alhajri.

TABLE I
FOCUS OF RELATED SURVEYS AND MONOGRAPHS. THE LAST COLUMN RECORDS WHETHER THE WORK COVERS THE LARGE-LEARNING-RATE (EDGE-OF-STABILITY) LITERATURE, WHICH BEGAN IN 2020–2021.

Work	Year	Main focus	EoS
Jain & Kar [16]	2017	structured non-convex problems; chapter on saddle escape	–
Bottou et al. [15]	2018	stochastic methods, noise reduction, second-order methods	–
Sun [18]	2020	practical optimization for deep learning; landscape results	–
Sun et al. [19]	2020	global landscape; spurious minima; width	–
Jin et al. [17]	2021	perturbed and stochastic gradient methods	–
Danilova et al. [20]	2022	complexity, variance reduction, adaptive methods	–
Ours	2026	saddle escape and large-step instability of unmodified gradient methods	✓

object, the instability of the GD map, and Ahn et al. [52] proved a single avoidance theorem that covers both. We use this common structure to present the two literatures side by side, and to ask what each can learn from the other.

A. Existing Surveys

Several excellent surveys address parts of this landscape, but each has a different focus (Table I). Bottou et al. [15] review stochastic optimization methods for large-scale machine learning, with an emphasis on convergence rates, noise reduction, and second-order methods (methods that use the Hessian) rather than on landscape geometry, the shape of the loss surface. Jain and Kar [16] provide a monograph on non-convex optimization for machine learning, centered on problems with known structure, such as recovering a sparse vector or a low-rank matrix from few measurements, with a chapter on escaping saddle points. Jin et al. [17] give a unified and rigorous treatment of perturbed gradient methods and their stochastic counterparts. Sun [18] offers a broad overview of optimization for deep learning, covering initialization, normalization, adaptive methods, and global landscape results. Sun et al. [19] survey the global landscape of neural networks, including results on spurious local minima (local minima that are not global) and the role of width. Danilova et al. [20] review recent theoretical advances in non-convex optimization, including first-order complexity, variance reduction (techniques that reduce gradient noise by occasionally computing full gradients), and adaptive methods.

These surveys were written before, or without a focus on, the large-learning-rate phenomena that now dominate the study of practical training: catapults, the edge of stability, self-stabilization, and their adaptive and stochastic extensions. None of them treats saddle escape and large-step instability as two faces of the same linear-stability question. Table I summarizes the focus of these works relative to ours.

B. Motivation, Scope, and Research Questions

The field as a whole spans critical-point geometry, saddle dynamics, escape algorithms, overparameterization, and practical training, and a single survey cannot treat all of it in depth. In this survey, we narrow the scope to the part of the field that concerns the dynamics of the gradient methods that are actually used, and we study it in more depth. The central question is: *what do unmodified gradient methods do when the GD map is locally unstable, and how does this instability help or hinder training?* We decompose it into two research questions (RQs):

- **RQ1 (Saddles).** When do unmodified GD and SGD avoid or escape saddle points, how long does escape take, and what roles do initialization and gradient noise play?
- **RQ2 (Stability).** What happens when the step size exceeds the classical stability limit $2/\lambda_{\max}(\nabla^2 L)$, and why does training still make progress?

Within scope are GD, SGD, momentum, and adaptive methods as used in practice, studied near saddles and at large step sizes. Algorithms designed specifically to escape saddles, such as cubic regularization, Hessian-free negative-curvature methods, and variance-reduced methods, which add second-order information or extra machinery to GD, are covered in [17], [20]; we discuss only perturbed GD [8], as the minimal modification that makes escape efficient. Overparameterized convergence theory is covered in [18], [19]. Generalization theory, which asks how well a trained network performs on new data, and the debate on whether flat minima generalize better than sharp ones [21]–[23], are discussed only where they bear directly on optimization.

C. Contributions

This survey makes the following contributions:

1) We organize the dynamics of unmodified gradient methods around a single object, the multiplier $1 - \eta\lambda$ of the GD map, and show how the saddle literature ($\lambda < 0$) and the large-step literature ($\lambda > 2/\eta$) answer the same questions differently (Section V, Table V).
2) We review what unmodified GD and SGD provably do near strict saddles: asymptotic avoidance, worst-case escape time, noise conditions for efficient escape, and saddle-to-saddle learning (Section III, Table III).
3) We review the large-step literature from its precursors to central flows and its adaptive and stochastic extensions, and collect the stability thresholds of the main optimizers in a single table (Section IV, Table IV).
4) We provide original simulations of saddle escape by GD, perturbed GD, SGD, and momentum, of saddle-to-saddle learning in linear networks, of the edge of stability in a small multilayer perceptron, and of a period-doubling route to chaos in a two-layer linear model.
5) We critically compare what is proved with what is observed, and derive open problems tied to specific gaps in the evidence (Sections V and VI).

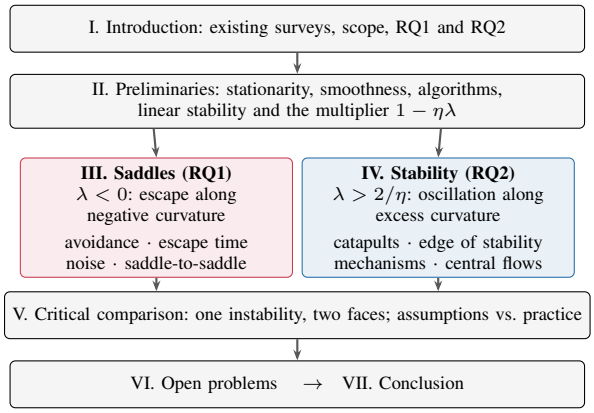


Fig. 1. Structure of this survey. The preliminaries introduce the multiplier $1-\eta\lambda$ of the gradient-descent map. Its two ways of exceeding one in magnitude define the two halves of the survey: negative curvature (saddle escape, Section III) and excess curvature (the edge of stability, Section IV). A cross-cutting analysis then compares them.

D. Organization

Section II builds the mathematical preliminaries: gradients, Hessians, stationarity notions, smoothness assumptions, the algorithms under study, the linear-stability threshold that connects the two halves of the survey, and the vocabulary of dynamical systems, noise, and curvature measurement that the later sections use. Section III answers RQ1 and Section IV answers RQ2. Section V compares the two literatures and their assumptions, Section VI discusses open problems and future directions, and Section VII concludes. Fig. 1 shows the structure of the survey.

How to read this survey. We wrote it to be readable by a student who knows only the background of a first course in machine learning: linear algebra, basic probability, and multivariable calculus, as in our MLR570 course. Section II builds everything else from that background, with derivations and small examples. Every technical term used in Sections III to VI is either built in Section II, with a pointer to the subsection, or explained in plain words where it first appears. The glossary in Appendix A lists every term with the place where it is defined.

II. Mathematical Preliminaries

This section builds every mathematical object that the rest of the survey uses. We assume only the background of our MLR570 course: linear algebra (eigenvalues and eigenvectors, the diagonalization of a symmetric matrix, norms, and the singular value decomposition), probability (expectation, variance, and covariance), and calculus (derivatives, gradients, the chain rule, the Hessian, convexity, and the gradient-descent update). Everything else is defined here, usually with a small worked example, before it is used. Later sections point back to the subsection where each idea is built. Table II summarizes the notation, and the glossary in Appendix A lists every technical term with the place where it is defined.

A. From a Neural Network to an Optimization Problem

Consider a training set $\{(x_i, y_i)\}_{i=1}^n$ of n inputs x_i and targets y_i, and a network $f(x;\theta)$ whose parameters, all weights and biases, are stacked into one vector $\theta \in \mathbb{R}^d$. Training minimizes the empirical risk

$$L(\theta) = \frac{1}{n}\sum_{i=1}^{n} \phi\big(f(x_i;\theta), y_i\big) = \frac{1}{n}\sum_{i=1}^{n} L_i(\theta), \qquad (1)$$

where ϕ is a per-example loss, such as the squared error $\phi(\hat{y}, y) = (\hat{y} - y)^2$ or the cross-entropy, and L_i is the loss on example i alone. In the course, ordinary least squares is the special case $f(x;\theta) = \theta^\top x$ with the squared error. There $L(\theta) = \frac{1}{n}\|X\theta - y\|^2$, whose Hessian $\frac{2}{n}X^\top X$ is positive semi-definite, so the problem is convex and every local minimum is global.

A neural network composes such linear maps with fixed non-linear functions called activations, applied entry by entry, for instance $\mathrm{ReLU}(z) = \max(0, z)$ or $\tanh(z)$. A network with one hidden layer of m units computes $f(x;\theta) = W_2\, a(W_1 x)$, where a is the activation, W_1 and W_2 are weight matrices, and m, the number of rows of W_1, is the width of the layer. A deep network stacks several such layers, and the number of layers is its depth. A linear network has no activations, $f(x) = W_H \cdots W_1 x$. It can only compute linear functions, yet its loss is still non-convex in the weights, because the weights are multiplied together. Even when ϕ is convex in the prediction f, the composite map $\theta \mapsto L(\theta)$ is generally non-convex. A network is overparameterized when it has many more parameters than it needs to fit the training data.

A simple symmetry argument shows why non-convexity is unavoidable. Permuting the hidden units of a layer, together with the corresponding rows and columns of the adjacent weight matrices, leaves the network function unchanged, and therefore leaves L unchanged. If $\theta^\star$ is a minimizer, so is every permuted copy $\pi(\theta^\star)$. If L were convex, the average $\frac{1}{2}(\theta^\star + \pi(\theta^\star))$ would also be a minimizer. But this average typically corresponds to a network whose hidden units have been blended together and computes a different, worse function. Hence the set of minimizers of a neural-network loss is typically non-convex, and so is L itself. The same symmetry produces many equivalent minima connected through regions of higher loss. These regions are natural places for saddle points to appear. A second symmetry, rescaling, acts on homogeneous activations such as ReLU: multiplying the incoming weights of a unit by $c > 0$ and its outgoing weights by $1/c$ leaves the function unchanged.

B. Gradients, Hessians, and Local Models

The gradient $\nabla L(\theta) \in \mathbb{R}^d$ collects the partial derivatives $\partial L/\partial\theta_j$, and the Hessian $\nabla^2 L(\theta) \in \mathbb{R}^{d\times d}$ collects the second derivatives $\partial^2 L/\partial\theta_j\partial\theta_k$. We measure the length of a vector by its Euclidean norm $\|v\| = \sqrt{v^\top v}$.

Taylor's theorem. For a function of one variable, a small change δ gives $f(x+\delta) \approx f(x) + f'(x)\delta + \frac{1}{2}f''(x)\delta^2$: the slope gives the linear change, and the second derivative gives the

TABLE II
SUMMARY OF NOTATION.

Symbol	Meaning
$\theta \in \mathbb{R}^d$	vector of all trainable parameters; d = their number
$L(\theta)$, $L_i(\theta)$	training loss; loss on example i ($i = 1, \dots, n$)
∇L, $\nabla^2 L$	gradient; Hessian (symmetric $d \times d$ matrix)
λ_j, v_j	j-th eigenvalue and unit eigenvector of $\nabla^2 L$
$\lambda_{\min}(\cdot), \lambda_{\max}(\cdot)$	smallest / largest eigenvalue
$S(\theta)$	sharpness, $\lambda_{\max}(\nabla^2 L(\theta))$ (Sec. II-I)
$\|v\|$, $\|A\|$	Euclidean length; spectral norm $\max_j \|\lambda_j\|$ of a symmetric A
$A \succeq 0$, $A \succ 0$	all eigenvalues ≥ 0 (PSD), all > 0 (PD); $A \succeq B$ means $A - B \succeq 0$
η, β	step size (learning rate); momentum parameter
$G(\theta)$	one GD step as a map, $\theta - \eta \nabla L(\theta)$ (Sec. II-G)
g_t, ξ_t	stochastic gradient at step t; its noise $g_t - \nabla L(\theta_t)$
σ^2	bound on the variance of the stochastic gradient
S, N	batch size; number of training examples (Sec. II-H)
ℓ, ρ	Lipschitz constants of ∇L and of $\nabla^2 L$
ϵ, γ	gradient and curvature tolerances
Δ	initial suboptimality $L(\theta_0) - \inf_\theta L(\theta)$
$O, \Omega, \tilde{O}$	growth rates (Sec. II-D)

bend. The same idea in d variables, for a small displacement $\Delta \in \mathbb{R}^d$, is the local model

$$L(\theta + \Delta) \approx L(\theta) + \nabla L(\theta)^\top \Delta + \tfrac{1}{2} \Delta^\top \nabla^2 L(\theta) \Delta. \qquad (2)$$

The error of this approximation is of third order in $\|\Delta\|$, so it is small for small steps. Differentiating (2) with respect to Δ, using the course identity $\nabla_x (x^\top A x) = (A + A^\top)x$, gives the matching model of the gradient,

$$\nabla L(\theta + \Delta) \approx \nabla L(\theta) + \nabla^2 L(\theta) \Delta. \qquad (3)$$

Section IV-D needs one more term of this expansion. It adds $\frac{1}{2}$ times the vector whose k-th entry is $\sum_{i,j} \frac{\partial^3 L}{\partial \theta_k \partial \theta_i \partial \theta_j} \Delta_i \Delta_j$, which we write $\frac{1}{2} \nabla^3 L(\theta)[\Delta, \Delta]$.

The first-order term of (2) explains gradient descent: choosing $\Delta = -\eta \nabla L(\theta)$ makes it equal to $-\eta \|\nabla L(\theta)\|^2 \leq 0$, so a small enough step decreases the loss. The second-order term describes curvature. Since $\nabla^2 L$ is symmetric, it can be diagonalized as in the course, $\nabla^2 L = Q \Lambda Q^\top$, with an orthonormal basis of eigenvectors $\{v_j\}$ (the columns of Q) and real eigenvalues $\{\lambda_j\}$ (the diagonal of Λ). Moving along an eigenvector, $\Delta = \epsilon v_j$, the quadratic term equals $\frac{1}{2} \epsilon^2 \lambda_j$. A positive eigenvalue means that the loss curves upward in that direction, and a negative eigenvalue means that it curves downward, a direction of negative curvature. Along any unit vector u, the curvature is the Rayleigh quotient $u^\top \nabla^2 L \, u = \sum_j \lambda_j (u^\top v_j)^2$. This is a weighted average of the eigenvalues, so it always lies between $\lambda_{\min}$ and $\lambda_{\max}$. At a point where the gradient vanishes, negative curvature is the only first- or second-order information that reveals a way downhill.

C. Critical Points and Notions of Stationarity

A point $\theta^\star$ is a local minimum if $L(\theta^\star) \leq L(\theta)$ for all θ in some neighborhood of $\theta^\star$, and a global minimum if this holds for all θ. The following definitions recur throughout the survey.

Definition 1 (Critical point)**.** A point θ is a critical (or first-order stationary) point if $\nabla L(\theta) = 0$. It is an ϵ-first-order stationary point (ϵ-FOSP) if $\|\nabla L(\theta)\| \leq \epsilon$.

As in the course, a critical point is a local minimum if $\nabla^2 L \succ 0$ there, a local maximum if $\nabla^2 L \prec 0$, and a saddle point if the Hessian has eigenvalues of both signs. When some eigenvalue is zero, the Hessian alone cannot tell.

Definition 2 (Strict and degenerate saddles)**.** A critical point that is neither a local minimum nor a local maximum is a saddle point. It is a strict saddle if $\lambda_{\min}(\nabla^2 L(\theta)) < 0$. A saddle at which $\nabla^2 L(\theta) \succeq 0$, so that no direction of negative curvature exists, is degenerate (or non-strict). Whether such a point is a minimum or a saddle can then be decided only by third- or higher-order derivatives.

A function is α-strongly convex on a region if $\nabla^2 L \succeq \alpha I$ there, with $\alpha > 0$: it curves upward by at least α in every direction, like a bowl that is never flat.

Definition 3 (Strict-saddle property [6])**.** A twice-differentiable function L is $(\alpha, \gamma, \epsilon, \delta)$-strict saddle if every point θ satisfies at least one of the following: (i) $\|\nabla L(\theta)\| \geq \epsilon$; (ii) $\lambda_{\min}(\nabla^2 L(\theta)) \leq -\gamma$; or (iii) θ is within distance δ of a local minimum $\theta^\star$, and L is α-strongly convex on the 2δ-neighborhood of $\theta^\star$.

Definition 3 says that every point is either steep, has a clear downhill curvature direction, or is already close to a good minimum. It excludes degenerate saddles and is the central structural assumption of the escape theory in Section III.

Definition 4 (Approximate second-order stationarity)**.** A point θ is an (ϵ, γ)-second-order stationary point ((ϵ, γ)-SOSP) if

$$\|\nabla L(\theta)\| \leq \epsilon \quad \text{and} \quad \lambda_{\min}\big(\nabla^2 L(\theta)\big) \geq -\gamma. \qquad (4)$$

Under a ρ-Lipschitz Hessian (Assumption 2 below), the standard choice is $\gamma = \sqrt{\rho \epsilon}$, and such a point is simply called an ϵ-SOSP [8], [35].

Where does $\sqrt{\rho \epsilon}$ come from? A negative curvature $-\gamma$ can be trusted only over a distance r on which the Hessian does not change much. Since the Hessian changes by at most ρr over a distance r, the useful distance is $r \approx \gamma / \rho$. Moving that far along the negative-curvature direction lowers the loss by about $\gamma r^2 = \gamma^3 / \rho^2$. Following a gradient of size ϵ over the same distance lowers it by about $\epsilon r = \epsilon \gamma / \rho$. The two gains are equal when $\gamma^2 = \rho \epsilon$. With this choice, both tests detect the same amount of possible progress, of order $\epsilon^{3/2} / \sqrt{\rho}$. If L is strict saddle with thresholds larger than these tolerances, every ϵ-SOSP must lie near a local minimum. This is the logic that turns SOSP-finding algorithms into local-minimum-finding algorithms. Three outcomes must not be confused: an FOSP has a small gradient; an SOSP additionally has no significant negative curvature; a global minimum has loss no larger than any other point. In general, reaching one does not guarantee the next.

Fig. 2 illustrates the four prototypical critical points. The degenerate saddle $L(\theta) = \theta_1^2 - \theta_2^4$ in panel (c) is the key warning example. Its Hessian at the origin is $\mathrm{diag}(2, 0) \succeq 0$,

so every second-order test is passed, yet the loss decreases along θ_2. Section III shows that such points arise naturally in deep networks. Fig. 3 summarizes how the notions relate.

D. Smoothness and Noise Assumptions

Convergence guarantees require quantitative regularity. A function h is Lipschitz with constant K if $\|h(a) - h(b)\| \le K\|a-b\|$ for all a, b: it never changes faster than rate K. The following assumptions apply this idea to the gradient and to the Hessian. Section V discusses how well they match practice.

Assumption 1 (Lipschitz gradient)**.** $\|\nabla L(\theta) - \nabla L(\theta')\| \le \ell\|\theta-\theta'\|$ for all θ, θ'. Equivalently, for twice-differentiable L, $-\ell I \preceq \nabla^2 L(\theta) \preceq \ell I$: no curvature, up or down, exceeds ℓ anywhere. Such an L is called ℓ-smooth.

Assumption 2 (Lipschitz Hessian)**.** $\|\nabla^2 L(\theta) - \nabla^2 L(\theta')\| \le \rho\|\theta-\theta'\|$ for all θ, θ': the curvature itself changes at a bounded rate ρ.

Assumption 3 (Stochastic gradients)**.** The algorithm has access to a random estimate $g(\theta;\xi)$ of the gradient, where ξ denotes the randomness, for example which examples were drawn. It is unbiased, $\mathbb{E}_\xi[g(\theta;\xi)] = \nabla L(\theta)$, and its variance is bounded, $\mathbb{E}_\xi\|g(\theta;\xi) - \nabla L(\theta)\|^2 \le \sigma^2$. The minibatch gradient of Section II-H is the canonical example.

Assumption 4 ((L_0, L_1)-smoothness [24])**.** $\|\nabla^2 L(\theta)\| \le L_0 + L_1\|\nabla L(\theta)\|$, so that curvature may grow with the gradient norm rather than being globally bounded.

Why ℓ matters: the descent lemma. Under Assumption 1, the quadratic term of (2) is at most $\frac{\ell}{2}\|\Delta\|^2$, and this upper bound is exact, not only approximate [27]: $L(\theta+\Delta) \le L(\theta) + \nabla L(\theta)^\top\Delta + \frac{\ell}{2}\|\Delta\|^2$. Put in the GD step $\Delta = -\eta\nabla L(\theta)$:

$$L(\theta - \eta\nabla L(\theta)) \le L(\theta) - \eta\Big(1 - \frac{\ell\eta}{2}\Big)\|\nabla L(\theta)\|^2. \quad (5)$$

For $\eta < 2/\ell$ the bracket is positive and every step lowers the loss. With $\eta = 1/\ell$ each step lowers it by at least $\|\nabla L\|^2/(2\ell)$. As long as the gradient is larger than ϵ, each step therefore gains at least $\epsilon^2/(2\ell)$, and the total gain cannot exceed Δ. So after at most $2\ell\Delta/\epsilon^2$ steps, GD must have visited an ϵ-FOSP. This is the $O(\ell\Delta/\epsilon^2)$ rate used throughout, and almost every saddle-escape theorem rests on (5) with $\eta \le 1/\ell$. Section IV shows that practical training runs far outside this regime.

Assumptions 1 and 2 underpin almost all complexity results for saddle escape. Assumption 4 was proposed after it was observed that the local smoothness of neural-network losses correlates with the gradient norm along training trajectories [24]. It underlies recent analyses of Adam and of tamed SGD [25], [26].

Growth rates. We write $a = O(b)$ if a is at most a constant times b, $a = \Omega(b)$ if it is at least a constant times b, and $\tilde{O}(\cdot)$ for $O(\cdot)$ with poly-logarithmic factors (powers of $\log d$, $\log(1/\epsilon)$) ignored. $\mathrm{poly}(d)$ means at most a fixed power of d; $e^{\Omega(d)}$ means exponential in d. A statement holds with probability at least $1-\delta$ when it can fail only with probability δ, and with high probability when δ can be made small at little cost.

E. The Algorithms Under Study

Gradient descent iterates

$$\theta_{t+1} = \theta_t - \eta\nabla L(\theta_t). \quad (6)$$

By the argument after (5), under Assumption 1 with $\eta = 1/\ell$ GD finds an ϵ-FOSP in $O(\ell\Delta/\epsilon^2)$ iterations [27], and this rate is optimal among first-order methods for that function class [28]. Full-batch GD uses the exact gradient over all n examples.

Stochastic gradient descent (SGD) [29] replaces $\nabla L(\theta_t)$ by a stochastic gradient $g_t = g(\theta_t;\xi_t)$, typically the minibatch gradient of Section II-H. Its step sizes η_t may stay constant or decay over time. The classical RobbinsMonro conditions $\sum_t \eta_t = \infty$ and $\sum_t \eta_t^2 < \infty$ (for example $\eta_t \propto 1/t$) let the steps travel any distance while the accumulated noise stays bounded. Under Assumptions 1 and 3, Ghadimi and Lan [30] showed that a randomly selected SGD iterate satisfies $\mathbb{E}\|\nabla L(\theta)\|^2 \le \epsilon^2$ after $O(\epsilon^{-4})$ stochastic gradient evaluations, a rate that cannot be improved for general stochastic first-order methods [31].

Momentum methods add a velocity term. The heavy-ball method [32] uses

$$\theta_{t+1} = \theta_t - \eta\nabla L(\theta_t) + \beta(\theta_t - \theta_{t-1}), \quad (7)$$

with $0 \le \beta < 1$, so each step also repeats a fraction β of the previous one. Nesterov's accelerated gradient does the same, but evaluates the gradient at the extrapolated point $\theta_t + \beta(\theta_t - \theta_{t-1})$.

Adaptive methods give every coordinate its own step size, built from past gradients. AdaGrad [33] divides each coordinate by the square root of the sum of its past squared gradients. RMSProp uses an exponential moving average instead, $v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$ (squares taken entry by entry), and steps $\theta_{t+1} = \theta_t - \eta\, g_t/(\sqrt{v_t} + \varepsilon)$, where ε is a small constant that prevents division by zero. Adam [34] adds momentum, $m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$, corrects both averages for their zero start, and steps $\theta_{t+1} = \theta_t - \eta\,\hat{m}_t/(\sqrt{\hat{v}_t} + \varepsilon)$. All of these can be written as gradient descent with a preconditioner P_t, a diagonal matrix built from past gradients:

$$\theta_{t+1} = \theta_t - \eta P_t^{-1} m_t. \quad (8)$$

Section II-I shows that their stability is then governed by the preconditioned Hessian $P_t^{-1}\nabla^2 L$. Two further updates appear in Section IV. Normalized GD divides the step by the gradient norm, $\theta_{t+1} = \theta_t - \eta\nabla L/\|\nabla L\|$, so every step has length η. Sharpness-aware minimization (SAM) takes its gradient at a deliberately worse nearby point. Its unnormalized form is $\theta_{t+1} = \theta_t - \eta\nabla L\big(\theta_t + \rho\nabla L(\theta_t)\big)$, where $\rho > 0$ is the radius of the perturbation.

Section III also names some other first-order methods, which we define here only in one line each. Mirror descent replaces the Euclidean distance in the step by a different measure of distance suited to the problem. The proximal point method takes the implicit step $\theta_{t+1} = \arg\min_\theta\{L(\theta) + \frac{1}{2\eta}\|\theta - \theta_t\|^2\}$. (Block) coordinate descent updates one coordinate, or one block of coordinates, at a time. Manifold GD runs gradient descent on a curved surface, such as the set of unit vectors.

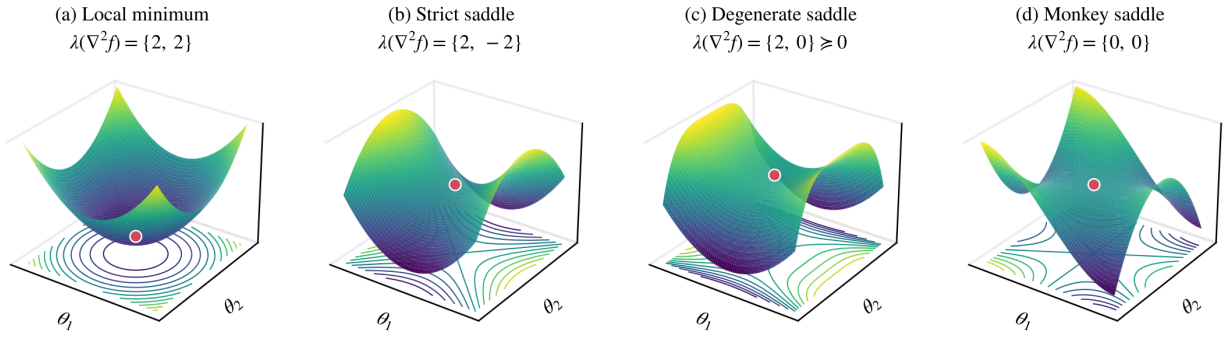


Fig. 2. The four prototypical critical points at the origin (red marker), with the Hessian spectrum above each surface. (a) A local minimum, $\theta_1^2 + \theta_2^2$. (b) A strict saddle, $\theta_1^2 - \theta_2^2$, with a direction of negative curvature that gradient methods can exploit. (c) A degenerate saddle, $\theta_1^2 - \theta_2^4$: the Hessian is positive semi-definite, so second-order information cannot reveal the descent direction along θ_2. (d) A monkey saddle, $\theta_1^3 - 3\theta_1\theta_2^2$, whose Hessian vanishes entirely. Section III shows that efficient escape guarantees exist for case (b) but not, in general, for (c) and (d).

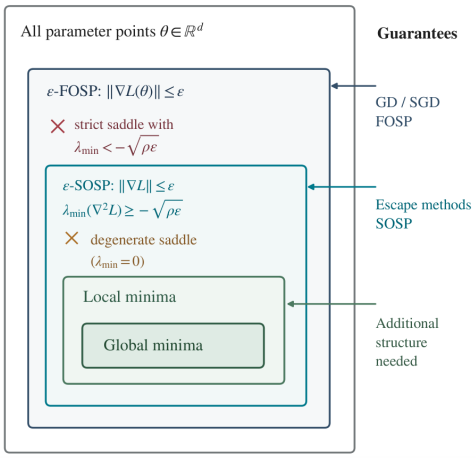


Fig. 3. Hierarchy of solution concepts. Under the relevant smoothness and noise assumptions, gradient descent and SGD guarantee membership in the outer set (ϵ-FOSP), which still contains strict saddles. Escape methods guarantee the ϵ-SOSP set, which still contains degenerate saddles. Only additional structure, such as the strict-saddle property, certifies local or global optimality.

Second-order methods use curvature directly. Newton's method takes $\Delta = -(\nabla^2 L)^{-1}\nabla L$, which by (3) jumps to the critical point of the local quadratic model. At a saddle point, this step is attracted towards the saddle rather than repelled, because it rescales a negative-curvature direction by a negative eigenvalue and so reverses its sign [5]. Cubic regularization [35] and related methods repair this, but they are outside our scope; the survey [17] treats them in detail.

Complexity results are stated in an oracle model. We count the calls to a gradient or stochastic-gradient oracle, a black box that returns $\nabla L(\theta)$ or $g(\theta;\xi)$ for a query point θ, needed to output an approximate FOSP or SOSP. This count ignores the cost of each call, memory, and parallelism.

F. Linear Stability and the $2/\eta$ Threshold

A single algebraic fact connects saddle escape (Section III) with the edge-of-stability phenomena of Section IV. In the course, gradient descent on $L(w) = w^\top w$ gave $w_{t+1} = (1-2\eta)w_t$, and the weights shrink only when $|1-2\eta| < 1$. The same calculation works near any critical point. Let $\theta^\star$ be a critical point with Hessian H. By (3) with $\theta = \theta^\star$ and $\nabla L(\theta^\star) = 0$, the gradient nearby is $\nabla L(\theta) \approx H(\theta - \theta^\star)$. Putting this into the GD step (6) and subtracting $\theta^\star$:

$$\theta_{t+1} - \theta^\star = (I - \eta H)(\theta_t - \theta^\star). \tag{9}$$

Write the error $\theta_t - \theta^\star$ in the eigenvectors of H, with coordinate u_j along v_j. Since $Hv_j = \lambda_j v_j$, the matrix $I - \eta H$ acts on each coordinate as multiplication by one number: $u_j \leftarrow (1-\eta\lambda_j)u_j$. We call $1-\eta\lambda_j$ the multiplier of direction j. After t steps, u_j has been multiplied by $(1-\eta\lambda_j)^t$. Two consequences follow.

Stability of minima. If $H \succ 0$, the iterates converge to $\theta^\star$ if and only if $|1-\eta\lambda_j| < 1$ for all j, that is,

$$\eta\,\lambda_{\max}(H) < 2. \tag{10}$$

A minimum whose sharpness $\lambda_{\max}(H)$ exceeds $2/\eta$ is linearly unstable under GD, regardless of how low its loss is. Section IV shows that practical training operates exactly at this boundary.

Escape from strict saddles. If $\lambda_j = -|\lambda_j| < 0$, the corresponding component grows by a factor $(1+\eta|\lambda_j|)$ per step, so after T steps it equals $\delta(1+\eta|\lambda_j|)^T$ if it started at δ. Here δ is the starting distance from the stable manifold, the set of points attracted to $\theta^\star$ (Section II-G). Solving $\delta(1+\eta|\lambda_j|)^T = r$ for T shows that GD needs

$$T_{\text{esc}} \approx \frac{\log(r/\delta)}{\log(1+\eta|\lambda_j|)} \tag{11}$$

iterations to travel a distance r away. The escape time is finite whenever $\delta > 0$, but it grows without bound as $\delta \to 0$, and it is long when the negative curvature is weak: for small $\eta|\lambda_j|$, $\log(1+\eta|\lambda_j|) \approx \eta|\lambda_j|$, so T_{esc} grows like $1/|\lambda_j|$. For example, with $\eta|\lambda_j| = 0.1$ and $r = 1$, every factor of 100 in δ costs $\log 100/\log 1.1 \approx 48$ extra steps. The whole saddle-escape literature can be read as an effort to control the two quantities in (11): to guarantee that δ is not too small, through randomness, and that $|\lambda_j|$ is not too small, through the strict-saddle property.

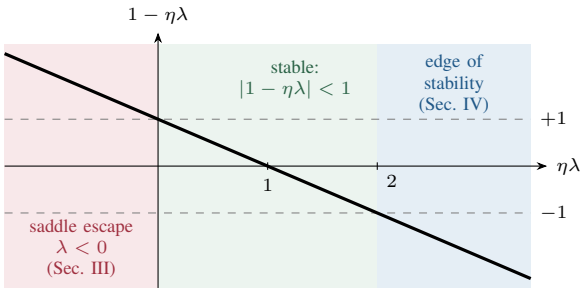


Fig. 4. The multiplier $1-\eta\lambda$ of the linearized GD map (9) along a Hessian eigenvector with eigenvalue λ. The component along that eigenvector decays only in the middle band, $0<\eta\lambda<2$. To the left, negative curvature makes the multiplier exceed $+1$, and GD leaves a strict saddle monotonically. To the right, excess curvature makes it fall below -1, and GD oscillates with growing amplitude, the starting point of catapults and the edge of stability.

Three regimes. Equation (9) therefore splits the spectrum into three regimes (Fig. 4). Directions with $0<\lambda_j<2/\eta$ contract, monotonically when $\lambda_j<1/\eta$ and with alternating sign when $1/\eta<\lambda_j<2/\eta$. Directions with $\lambda_j<0$ expand monotonically. Directions with $\lambda_j>2/\eta$ expand with alternating sign, so that the iterate oscillates across the valley with growing amplitude. Section III studies the first kind of instability and Section IV the second. In both cases, the instability is local. It says that GD cannot remain near the critical point, not where it will go next, and the higher-order terms that (9) discards decide what happens once the iterate has moved away. For the saddle, they determine where the escaping trajectory goes. For the edge of stability, they produce the self-stabilizing feedback described in Section IV-D.

G. Gradient Descent as a Dynamical System

Section III studies GD with the vocabulary of dynamical systems, the mathematics of repeated maps. We introduce it with one running example, the strict saddle

$$L_{\text{toy}}(\theta)=\tfrac{1}{2}\theta_1^2-\tfrac{1}{2}\theta_2^2+\tfrac{1}{4}\theta_2^4, \tag{12}$$

which has a strict saddle at the origin, with curvatures $+1$ along θ_1 and -1 along θ_2, and minima at $(0,\pm 1)$.

Maps and fixed points. One GD step is a map $G(\theta)=\theta-\eta\nabla L(\theta)$, and the iterates are $\theta_t=G(G(\cdots G(\theta_0)))$, written $G^t(\theta_0)$. A fixed point satisfies $G(\theta)=\theta$, which happens exactly when $\nabla L(\theta)=0$: the fixed points of GD are the critical points of L. The Jacobian of G (the course's matrix of first derivatives of a vector map) is $DG(\theta)=I-\eta\nabla^2 L(\theta)$. Its eigenvalues are the multipliers $1-\eta\lambda_j$ of Section II-F. A fixed point is unstable if some multiplier has magnitude larger than one. For (12), the step multiplies θ_1 by $1-\eta$ and maps θ_2 to $\theta_2+\eta(\theta_2-\theta_2^3)$.

Stable sets and measure zero. The stable set of a fixed point $\theta^\star$ is the set of starting points from which GD converges to it, $\{\theta_0 : G^t(\theta_0)\to\theta^\star\}$. For (12), near the saddle it is the line $\theta_2=0$: a start with $\theta_2=0$ keeps $\theta_2=0$ forever and slides into the saddle, while a nearby start with $\theta_2\neq 0$ is pushed away. (With a finite step, a few more lines belong to it, those that one GD step maps exactly onto $\theta_2=0$, such as $\theta_2=\pm\sqrt{1+1/\eta}$, and their own preimages. They too have zero area.) Near a fixed point, the stable set is typically a smooth curve or surface, called the stable manifold. A set has (Lebesgue) measure zero if it has zero length in $\mathbb{R}$, zero area in $\mathbb{R}^2$, or zero volume in $\mathbb{R}^d$. A line in the plane and a surface in space both have measure zero. A random starting point drawn from any distribution with a density, such as a Gaussian or a uniform distribution on a box, lands in a measure-zero set with probability zero. An event that has probability one is said to happen almost surely (a.s.).

The stable-manifold theorem. The center-stable manifold theorem of dynamical systems [36] says that, near a fixed point of a smooth invertible map where some multiplier has magnitude larger than one, the points whose iterates stay nearby forever lie on a smooth surface of dimension strictly less than d. This surface is tangent to the directions whose multipliers have magnitude at most one. In other words, it has measure zero. The theorem needs G to be a diffeomorphism, a smooth map with a smooth inverse, so that it never folds a region of positive volume onto a set of zero volume. The GD map is one when $\eta<1/\ell$. Then every $|\eta\lambda_j|<1$, so every factor of $\det DG=\prod_j(1-\eta\lambda_j)$ is positive and DG is invertible everywhere. Section III assembles these facts into a proof that GD avoids strict saddles.

Gradient flow. If the step size is small, many GD steps move θ only a little. In the limit $\eta\to 0$, with time measured as $s=\eta t$, GD becomes the differential equation

$$\frac{d\theta}{ds}=-\nabla L(\theta), \tag{13}$$

called gradient flow. Counting time in steps instead, the same equation reads $d\theta/dt=-\eta\nabla L(\theta)$, the form used in Section IV. Along it the loss always decreases, since by the chain rule $\frac{dL}{ds}=\nabla L^\top\frac{d\theta}{ds}=-\|\nabla L\|^2\le 0$. GD follows gradient flow closely when all multipliers lie in $(0,1)$, that is, when the step is small compared with the curvature. Many analyses study gradient flow because it is easier to solve exactly.

Invariant sets. A set $\mathcal{S}$ is (forward) invariant if the dynamics never leave it once inside: $G(\mathcal{S})\subseteq\mathcal{S}$. The line $\theta_2=0$ of (12) is invariant. In networks, the symmetries of Section II-A create invariant sets: two hidden units with identical weights receive identical gradients and stay identical forever. When such a set is a smooth surface, it is called an invariant manifold.

Oscillations and period doubling. GD need not converge to a point. A period-two orbit is a pair of points $a\neq b$ with $G(a)=b$ and $G(b)=a$: the iterates bounce between them forever. For example, on $f(x)=\frac{\lambda}{2}x^2$ with $\eta\lambda=2$, the step is $x\mapsto(1-2)x=-x$, and every starting point bounces between x_0 and $-x_0$. As the step size crosses a critical value, a stable fixed point can lose its stability while a stable period-two orbit appears. This qualitative change is called a bifurcation. Repeating it gives orbits of period 4, 8, and so on, a period-doubling cascade, and eventually chaos: irregular iterates that never repeat.

Time averages and projections. When iterates oscillate, it can be easier to follow their time average over a short window than the iterates themselves. One further tool from linear algebra, the projection used in the course's geometry of least

squares, appears there. For a nonzero vector v, the matrix $\Pi_v^\perp = I - vv^\top/\|v\|^2$ removes the component of any vector along v. Moving in the direction $-\Pi_v^\perp \nabla L$ therefore follows the gradient as closely as possible while keeping unchanged, to first order, any quantity whose gradient is v.

H. Randomness: Minibatch Noise and Its Shape

Minibatches. With N training examples, a minibatch is a random subset B of S of them, and the minibatch gradient is $g = \frac{1}{S}\sum_{i\in B} \nabla L_i(\theta)$. Because each example is equally likely to be drawn, linearity of expectation gives $\mathbb{E}[g] = \nabla L(\theta)$: the minibatch gradient is unbiased. Its noise is $\xi = g - \nabla L(\theta)$, a random vector with mean zero.

The shape of the noise. The noise is described by its covariance matrix $\Sigma = \mathbb{E}[\xi\xi^\top]$, as in the course. The variance of the noise along a unit direction v is $v^\top \Sigma v$. Noise is isotropic if $\Sigma = cI$, equally strong in every direction, and anisotropic otherwise. Real SGD noise is anisotropic, and its shape is related to the Hessian. Isotropic noise has a weakness in high dimension. If u is drawn uniformly from the unit sphere in $\mathbb{R}^d$, then $\sum_j u_j^2 = 1$ and, by symmetry, every $\mathbb{E}[u_j^2]$ is the same, so $\mathbb{E}[u_j^2] = 1/d$. More generally, $\mathbb{E}[(u^\top v)^2] = 1/d$ for any fixed unit vector v. The noise along any one direction, such as the single escape direction of a saddle, therefore has typical size only $1/\sqrt{d}$.

Batch size. Averaging over a larger batch reduces the noise. For a quantity h_i that varies across examples with mean $\bar{h}$ and variance $s^2 = \frac{1}{N}\sum_i (h_i - \bar{h})^2$, the average over a batch of S examples drawn without replacement has variance

$$\mathrm{Var}\Big(\frac{1}{S}\sum_{i\in B} h_i\Big) = \frac{s^2}{S}\cdot\frac{N-S}{N-1}, \tag{14}$$

the standard variance of a sample mean with a finite-population correction. It is s^2 for $S = 1$ and 0 for the full batch $S = N$. Section IV applies (14) to the per-example curvatures.

Heavy tails. A random variable X is heavy-tailed if large values are not rare: $P(|X| > t)$ decays only like a power $t^{-\alpha}$, instead of like $e^{-t^2/2}$ as for a Gaussian. The exponent α is the tail index; a smaller α means heavier tails. If $\alpha \le 2$, the variance is infinite, and Assumption 3 fails.

Rare events. Large-deviations theory studies the probability of events that need a long run of unlikely noise, such as SGD climbing over a barrier of height E. For small step sizes such probabilities decay exponentially, like $e^{-E/\eta}$ up to constants, so the expected waiting time for the event grows like $e^{E/\eta}$.

I. Measuring Curvature in Practice

Sharpness. The sharpness of the loss at θ is its largest Hessian eigenvalue, $S(\theta) = \lambda_{\max}(\nabla^2 L(\theta))$, the curvature of its steepest direction. At a minimum it is the sharpness of the minimum in (10). Away from minima it is a function of θ that changes during training. Section IV names its typical behaviors: progressive sharpening, when S rises during training, and the edge of stability, when S stays near $2/\eta$. When the largest eigenvalue is simple (not repeated), with unit eigenvector u, then $S(\theta) = u^\top \nabla^2 L(\theta) u$. Its gradient is obtained by differentiating the Hessian while keeping u fixed, $\nabla S(\theta) = \nabla^3 L(\theta)[u, u]$, with the third-derivative notation of Section II-B. The vector ∇S points in the direction in which the loss becomes sharper fastest.

Hessian-vector products and power iteration. For a network with millions of parameters, the Hessian cannot be stored. But its product with a vector can be computed. By (3), $\nabla^2 L(\theta)v \approx (\nabla L(\theta + hv) - \nabla L(\theta))/h$ for a small h, and automatic differentiation computes it exactly at the cost of a few gradients. This is a Hessian-vector product (HVP). Repeating $v \leftarrow \nabla^2 L\, v/\|\nabla^2 L\, v\|$ is power iteration. Writing v in the eigenvectors, each step multiplies the component along v_j by λ_j, so after k steps the components are in the ratio λ_j^k. The vector therefore turns towards the eigenvector with the largest $|\lambda_j|$, and $v^\top \nabla^2 L\, v$ converges to that eigenvalue. This is how the sharpness of real networks is measured.

Conditioning. The condition number of a positive definite matrix is $\kappa = \lambda_{\max}/\lambda_{\min}$. A large κ means a long, narrow valley, and an ill-conditioned problem. A matrix is well-conditioned when κ is small.

Preconditioned curvature. For the preconditioned update (8), the same linearization as in (9) gives the matrix $I - \eta P^{-1} H$. Its stability is therefore decided by the eigenvalues of the preconditioned Hessian $P^{-1}H$ instead of those of H.

Singular values. For a non-square matrix, such as the end-to-end map $W = W_H \cdots W_1$ of a linear network or the input-output correlation matrix of a data set, the course's singular value decomposition $W = U\Sigma_W V^\top$ plays the role of the eigendecomposition. The singular values on the diagonal of Σ_W measure how strongly W maps each input direction to its output direction. The rank of W is the number of nonzero singular values. A data set has whitened inputs when the inputs have been linearly transformed so that their covariance is the identity, for instance through PCA as in the course.

III. Escaping Saddles with Unmodified Gradient Methods (RQ1)

This section asks what unmodified GD and SGD do near strict saddles, and how long they spend there. Keep the toy saddle (12) in mind. A ball started exactly on the line $\theta_2 = 0$ rolls into the saddle and stops; a ball started anywhere else eventually rolls off along θ_2. The questions are how likely the first case is, how long "eventually" takes, and what noise changes. Section III-A explains why saddles matter, Section III-B shows that GD almost never stops at one, Section III-C shows that it can still wait very long, Section III-D shows how noise helps, and Section III-E shows that saddles can be stages of learning. Table III summarizes the main results.

A. Why Saddles Matter

Dauphin et al. [5] gave the saddle question its modern form. Drawing on results from statistical physics for random Gaussian fields (random functions whose values at any finite set of points are jointly Gaussian), they argued that for a random function in high dimension d, a critical point is a

local minimum only if all d Hessian eigenvalues are positive. If each sign were an independent coin flip, this would happen with probability 2^{-d}. More precisely, the index of a critical point, the fraction of negative Hessian eigenvalues, is strongly correlated with its loss value. Critical points with high loss are overwhelmingly saddles, and local minima concentrate at loss values close to the global minimum. Dauphin et al. confirmed this correlation empirically for small networks on MNIST and CIFAR-10, standard benchmark data sets of handwritten digits and of small natural images. They also showed that the plateaus surrounding saddles, regions where the loss is almost flat, slow down gradient descent. If bad local minima are rare, the practical obstacle is the time spent near saddles, and the key quantity becomes the curvature of the escape direction, as in (11).

Depth makes this obstacle worse. For deep linear networks (Section II-A), Kawaguchi [37] proved, under full-rank conditions on the data and distinct eigenvalues, that for any depth and any widths every local minimum is a global minimum and every other critical point is a saddle. At saddles where the product of the hidden-layer weights has full rank, the Hessian has at least one strictly negative eigenvalue. But networks with two or more hidden layers also possess "bad" saddles whose Hessian has no negative eigenvalue at all, such as the origin. To see why, write the loss as $\frac{1}{2}\|Y - W_3W_2W_1X\|^2$ for a network with three weight matrices. Every second derivative of this loss still contains at least one weight matrix as a factor, so at the origin, where all weights are zero, the Hessian is the zero matrix. With only two weight matrices, the mixed derivative $\partial^2 L/\partial W_1 \partial W_2$ contains the data term $YX^\top$ instead, and the origin is a strict saddle. Minima are benign, but some saddles are degenerate, and the escape theory below does not cover them.

B. Asymptotic Avoidance: GD Almost Never Converges to a Strict Saddle

The foundational result is due to Lee et al. [7].

Theorem 1 (Lee et al. [7], informal)**.** Let L be twice continuously differentiable with an ℓ-Lipschitz gradient, let $0 < \eta < 1/\ell$, and suppose every saddle point of L is strict. If θ_0 is drawn from any distribution with a density, then GD (6) converges to a strict saddle with probability zero. Consequently, whenever GD converges, it converges to a local minimizer almost surely.

The proof of Theorem 1 is a short and elegant use of the dynamical-systems vocabulary of Section II-G. The GD update $G(\theta) = \theta - \eta\nabla L(\theta)$ has Jacobian $DG = I - \eta\nabla^2 L(\theta)$, which is invertible when $\eta < 1/\ell$, so G is a diffeomorphism. At a strict saddle, DG has an eigenvalue $1 + \eta|\lambda_{\min}| > 1$, the left region of Fig. 4. By the center-stable manifold theorem [36], the points that converge to this saddle lie, locally, on a manifold of dimension strictly less than d, and such a manifold has Lebesgue measure zero. The set of starting points that eventually reach this neighborhood is obtained by pulling the small set back through G again and again (its preimages). Since preimages of measure-zero sets under a diffeomorphism also have measure zero, the global set of initializations that converge to any strict saddle is negligible.

This argument has been extended in several directions. Lee et al. [38] generalized it into a single theorem: for any first-order method whose update map g has an invertible Jacobian everywhere, the set of initial points that converge to an unstable fixed point of g (a fixed point with a multiplier of magnitude larger than one) has measure zero. Their proof covers all saddles by countably many small neighborhoods, which removes the need for the saddles to be isolated, that is, separated from one another. They then verified the conditions for GD, the proximal point method, coordinate and block coordinate descent, manifold gradient descent, and mirror descent (all defined in Section II-E). Panageas et al. [39] proved a stable-manifold theorem for time-varying dynamical systems, in which the map changes from step to step as the step size changes, and showed that GD, mirror descent, the proximal point method, and manifold gradient descent avoid strict saddles almost surely under vanishing step sizes decaying no faster than $\Omega(1/k)$; the corrected version of their proof assumes that the Hessian at the saddle is invertible. They also noted that step sizes that decay too quickly, so that their sum is finite, can stop the iterates at points that are not even critical: the steps run out before the iterate arrives. Antonakopoulos et al. [40] showed that the AdaGrad family, with scalar, diagonal, or full-matrix preconditioning (one step size for all coordinates, one per coordinate, or a full matrix), avoids saddle points from almost every initialization when the gradients are exact. This was a non-trivial extension, because AdaGrad's preconditioner depends on the entire gradient history, so earlier avoidance results for constant or vanishing step sizes do not apply directly. For stochastic methods, Mertikopoulos et al. [41] showed that SGD with step sizes decaying as $1/n^p$, a family that includes but goes beyond the Robbins–Monro schedule (Section II-E), converges almost surely to the set of critical points. Under bounded and uniformly exciting noise, meaning noise of bounded size that has a nonzero component in every direction, it avoids strict saddles with probability one. This includes non-isolated saddle manifolds, continuous curves or surfaces made entirely of saddles. Apidopoulos et al. [26] extended almost-sure convergence and strict-saddle avoidance to "tamed" variants of SGD, which shrink the step where the loss is steep, under the (L_0, L_1)-smoothness of Assumption 4, removing the globally Lipschitz gradient that the classical results require. Finally, Cheridito et al. [42] proved a center-stable manifold theorem that needs less smoothness than the classical one, which matters because ReLU is not differentiable at zero. They applied it to networks with one hidden layer of ReLU or leaky-ReLU units ($\max(z, cz)$ for a small $c > 0$) and a one-dimensional input, trained to fit a straight-line target $ax + b$ under the squared error integrated over the input interval. They showed that GD avoids most saddle points, and that if the loss along the GD trajectory converges to a value below an explicit threshold, then the limit is a global minimum.

Together, these results settle the asymptotic question under strict-saddle conditions. They say nothing about how long convergence takes, and nothing about degenerate saddles,

which the stable-manifold argument does not cover.

C. Avoidance Is Not Efficiency

The escape-time formula (11) already suggests why avoidance and efficiency differ: an initialization at distance δ from the stable manifold escapes in time proportional to $\log(1/\delta)$, which is finite but unbounded. Du et al. [43] turned this observation into a striking negative result. They constructed smooth functions on $\mathbb{R}^d$ containing a chain of saddle points, arranged so that the trajectory exiting one saddle arrives very close to the stable manifold of the next. While GD escapes one saddle, the escape direction of the next saddle still curves upward, so the offset along it shrinks, and each wait is longer than the last by a constant factor. The waits multiply. Under fairly natural random initialization schemes, GD then takes time exponential in d, $e^{\Omega(d)}$ steps, to escape the sequence of saddles, whereas perturbed GD (below) escapes in time polynomial in d. Almost-sure avoidance is therefore compatible with arbitrarily slow escape.

The minimal fix is to add randomness when it is needed. Perturbed GD (PGD) [8] runs GD and, whenever the gradient is small and no perturbation has been added recently, adds a perturbation drawn uniformly from a small ball. If the loss has not decreased enough after a fixed number of further steps, PGD stops and returns the point before the perturbation: since a random kick did not lead downhill, there was no significant negative curvature there, and the point is an approximate SOSP. Jin et al. proved that, under Assumptions 1 and 2, PGD outputs an ϵ-SOSP with high probability in $O\big(\ell\Delta/\epsilon^2 \cdot \log^4(d\ell\Delta/(\epsilon^2\delta))\big)$ iterations, where $1-\delta$ is the success probability. This matches the first-order rate $O(\ell\Delta/\epsilon^2)$ of GD (Section II-D) up to poly-logarithmic factors in d, so escaping strict saddles is almost free in iteration count. The key geometric fact is that the perturbations that fail to escape form a thin region around the stable manifold, so a random perturbation leaves it with high probability.

Fig. 5 illustrates the mechanism on the two-dimensional function $L(\theta) = \frac{1}{2}\theta_1^2 - \frac{1}{2}\theta_2^2 + \frac{1}{4}\theta_2^4$, which has a strict saddle at the origin and minima at $(0, \pm 1)$. Started at $(1.5, 10^{-7})$, very close to the stable manifold $\{\theta_2 = 0\}$, GD first converges towards the saddle and then waits on a plateau for roughly 150 iterations before escaping. Panel (c) confirms the logarithmic law (11): the GD escape time grows linearly in $\log(1/\delta)$. In contrast, the median escape time of perturbed GD is essentially independent of δ, because a single random perturbation places the iterate at a typical distance from the stable manifold.

D. Noise as an Escape Mechanism

Deterministic GD needs luck in its initialization to escape quickly. SGD has a built-in source of randomness, and a central question is whether that randomness is enough.

Isotropic noise. Ge et al. [6] gave the first polynomial-time guarantee. Under the strict-saddle property, noisy SGD, which adds noise drawn uniformly from the unit sphere to each stochastic gradient, reaches a neighborhood of a local minimum in a number of iterations polynomial in d and the problem parameters. The authors noted that the added noise is not needed if the stochastic gradients already have non-negligible variance in every direction. Isotropic noise guarantees a component along the escape direction, but its strength along any fixed direction scales as $1/\sqrt{d}$ (Section II-H), so the guarantees inherit a polynomial dependence on dimension.

Correlated negative curvature. Daneshmand et al. [44] asked whether the noise of plain SGD already has the required component. They proposed the correlated negative curvature (CNC) condition: there exists $\gamma > 0$ such that, for all θ,

$$\mathbb{E}_\xi\big[\langle v_\theta, g(\theta;\xi)\rangle^2\big] \geq \gamma, \tag{15}$$

where v_θ is a unit eigenvector for the smallest eigenvalue $\lambda_{\min}(\nabla^2 L(\theta))$, the most negative curvature direction. In words, the stochastic gradients have a spread of at least γ along the way down. They showed empirically that, for several models including neural networks, the variance of stochastic gradients along negative-curvature directions does not shrink with dimension, whereas isotropic noise does. Under CNC, a variant of SGD that periodically takes a larger step reaches an $(\epsilon, \epsilon^{2/5})$-SOSP in $\tilde{O}(\epsilon^{-4})$ iterations with high probability, and a variant that runs GD and takes a single, larger stochastic-gradient step whenever the gradient is small does so in $\tilde{O}(\epsilon^{-2})$ iterations. Neither rate depends explicitly on d. Fang et al. [45] sharpened this picture with a dispersive noise assumption: the noise does not concentrate its probability in narrow regions. Under it, and with Lipschitz gradients and Hessians, SGD with a simple averaging and restart rule finds an ϵ-SOSP in $\tilde{O}(\epsilon^{-3.5})$ stochastic gradients. This improves on the $\tilde{O}(\epsilon^{-4})$ rate of perturbed SGD with isotropic noise [17], without injecting any noise.

Heavy tails and global exploration. Gradient noise in deep learning is often far from Gaussian. Gürbüzbalaban et al. [46] showed that heavy-tailed behavior (Section II-H) emerges in the SGD iterates even for linear regression with Gaussian data. The reason is that the noise is multiplicative: each SGD step on least squares multiplies the current iterate by a random matrix, and long products of random matrices produce heavy tails. The tail index depends on the step size, the batch size, the data variance, and the dimension, with larger step sizes and smaller batches producing heavier tails. Heavy tails mean occasional large jumps, which can move the iterate between basins. Azizian et al. [47] quantified such global behavior using large-deviations theory (Section II-H). Modeling SGD as a randomly perturbed dynamical system, they derived matching upper and lower bounds on the expected time to reach the global minimizers. This time scales as $\exp(E/\eta)$, where the energy E is determined by the barriers that SGD might have to climb on its way to the global minimum. In landscapes without spurious minima the energy vanishes, and the expected time grows more slowly than any such exponential. Spurious minima do not merely exist, they control how long noisy training takes.

These analyses show that noise can provide escape without deliberate perturbation, but only under assumptions such as CNC or dispersivity on its geometry relative to the Hessian. Whether minibatch noise in modern architectures satisfies such conditions throughout training is an empirical question that remains largely open (Section VI).

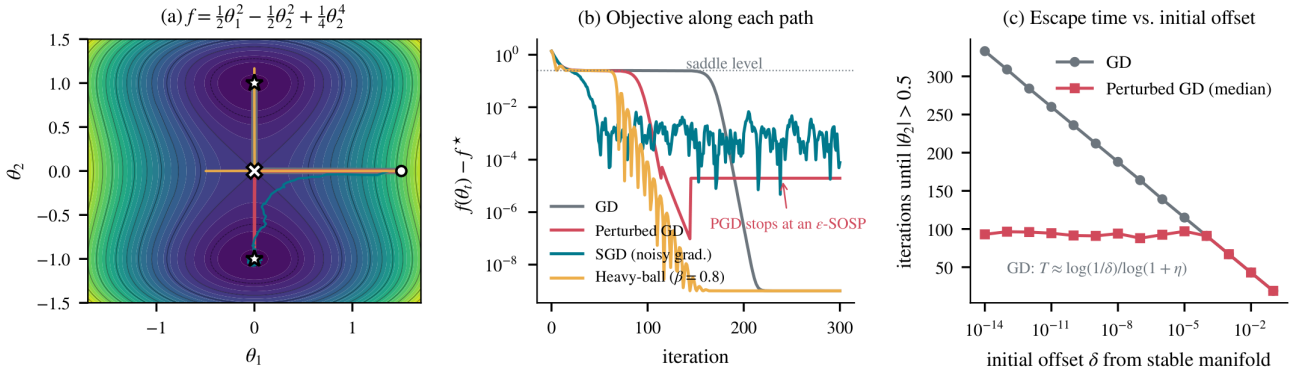


Fig. 5. Original simulation of saddle escape on $L = \frac{1}{2}\theta_1^2 - \frac{1}{2}\theta_2^2 + \frac{1}{4}\theta_2^4$ (strict saddle at the origin, marked $\times$; minima at $(0, \pm 1)$, marked $\star$). All methods use $\eta = 0.1$ and start at $(1.5, 10^{-7})$. (a) Trajectories over the loss contours. (b) Suboptimality over iterations. GD lingers at the saddle level; perturbed GD [8] escapes as soon as the gradient is small and stops at an ϵ-SOSP once a perturbation no longer decreases the loss; SGD with isotropic gradient noise escapes quickly but stalls at a noise floor; heavy-ball momentum [32] escapes earliest. (c) Iterations needed to reach $|\theta_2| > 0.5$ as a function of the initial distance δ from the stable manifold: GD follows the $\log(1/\delta)$ law of (11), whereas perturbed GD is insensitive to δ.

E. Plateaus and Saddle-to-Saddle Dynamics

Escape time also matters at initialization. Neural networks are typically initialized with small weights, and the origin of weight space is a critical point of most architectures, for example of every deep linear network. Training from small initialization therefore begins near a saddle.

Saxe et al. [48] derived exact solutions for this regime in deep linear networks with whitened inputs (Section II-I). Let the input-output correlation matrix have singular value decomposition $\Sigma_{yx} = USV^\top$ with singular values $s_1 > s_2 > \cdots$. Each singular value is the strength of one input-output pattern of the data, a mode. For a network with one hidden layer whose weights start aligned with these singular vectors, learning decouples into independent modes. The strength a_k of mode k, the product of the corresponding input and output weights, evolves according to

$$\tau \frac{da_k}{dt} = 2a_k(s_k - a_k), \qquad a_k(t) = \frac{s_k e^{2s_k t/\tau}}{e^{2s_k t/\tau} - 1 + s_k/a_k(0)}, \tag{16}$$

where τ is the inverse learning rate in continuous time (gradient flow, Section II-G). This is the logistic equation, and the formula on the right is its exact solution, which can be checked by differentiating it. Each mode follows a sigmoidal curve. While a_k is small, the equation reads $\tau\, da_k/dt \approx 2s_k a_k$, so $a_k(t) \approx a_k(0)e^{2s_k t/\tau}$: exponential growth from the saddle, exactly as in (11). Reaching a size of order s_k therefore takes a time of order $(\tau/2s_k)\log(s_k/a_k(0))$, after which the mode rises rapidly to s_k and stops. Stronger modes are learned first, and the smaller the initialization, the longer the plateaus. Saxe et al. also showed that random orthogonal initializations can make learning time nearly independent of depth. This is an early form of the dynamical-isometry principle: if all singular values of the product of the layer matrices are close to one, signals neither explode nor vanish as they pass through many layers.

Equation (16) produces saddle-to-saddle dynamics. The network sits near the saddle at the origin, escapes along the direction of the strongest mode, then lingers near a new saddle that corresponds to a rank-one solution, and so on. Fig. 6 reproduces this staircase in simulation. Zhang et al. [14] recently placed this phenomenon in a general framework covering fully connected networks (dense layers), convolutional networks (layers of small filters slid over an image), and attention-based architectures (the layers of Transformers, made of parallel units called heads). They showed that the fixed points and invariant manifolds (Section II-G) of a wide network include those of narrower networks: a wide network whose extra units are zero or duplicated behaves exactly like a narrower one, so that gradient descent iteratively evolves near an invariant manifold, approaches a saddle, and switches to the next manifold. As a result, linear networks learn solutions of increasing rank, ReLU networks learn solutions with an increasing number of kinks (points where the output changes slope), convolutional networks learn an increasing number of kernels, and attention models learn an increasing number of heads. The framework separates data-induced saddle-to-saddle dynamics, ordered by the strength of the data's modes, which lead to low-rank weights (few nonzero singular values), from initialization-induced dynamics, ordered by which unit happened to start largest, which lead to sparse weights (few active units), and it predicts how the data distribution and initialization affect the number and duration of plateaus. For shallow ReLU-like networks, Wu et al. [49] refined the description of the saddle-to-saddle process when the initialization scale tends to zero, by ruling out saddle-escape types that earlier analyses had not excluded.

This line of work changes the meaning of a saddle. In the escape literature, a saddle is an obstacle to be left as quickly as possible. In the saddle-to-saddle view, the sequence of saddles visited is the mechanism by which networks acquire progressively more complex functions, a form of simplicity bias: the network fits the strongest, simplest pattern first and adds detail later. An optimizer that escaped every saddle instantly might alter which solution is found, a tension we revisit in Section VI.

TABLE III
REPRESENTATIVE RESULTS ON SADDLE AVOIDANCE AND ESCAPE BY UNMODIFIED OR MINIMALLY MODIFIED GRADIENT METHODS. "A.S.": ALMOST SURELY; SOSP: SECOND-ORDER STATIONARY POINT (DEFINITION 4).

Work	Method	Key assumptions	Guarantee
Lee et al. [7]	GD	C^2, ℓ-Lipschitz gradient, $\eta < 1/\ell$, random init. with a density	converges to a strict saddle with probability zero (asymptotic)
Lee et al. [38]	GD, proximal point, (block) coordinate descent, mirror and manifold GD	update map with invertible Jacobian; saddles need not be isolated	stable set of strict saddles has measure zero
Panageas et al. [39]	GD and variants, vanishing steps	steps $\to 0$, $\Omega(1/k)$; invertible Hessian at the saddle (corrected proof)	a.s. avoidance; summable steps may stop at non-critical points
Antonakopoulos et al. [40]	AdaGrad (scalar, diagonal, full)	deterministic gradients	a.s. avoidance of strict saddles
Mertikopoulos et al. [41]	SGD, steps $\propto 1/n^p$	bounded, uniformly exciting noise	a.s. avoidance, incl. non-isolated saddle manifolds
Apidopoulos et al. [26]	tamed SGD	(L_0, L_1)-smoothness	a.s. convergence and strict-saddle avoidance
Cheridito et al. [42]	GD, shallow (leaky) ReLU	scalar input, affine target	avoids most saddles; global min. if the limit loss is below a threshold
Du et al. [43]	GD	constructed chain of saddles	escape can take time exponential in d
Jin et al. [8]	perturbed GD	Lipschitz gradient and Hessian	ϵ-SOSP in $\tilde{O}(\ell\Delta/\epsilon^2)$, poly-log in d
Ge et al. [6]	noisy SGD	$(\alpha, \gamma, \epsilon, \delta)$-strict saddle	near a local minimum in poly(d) iterations
Daneshmand et al. [44]	SGD with occasional larger steps	CNC (15)	$(\epsilon, \epsilon^{2/5})$-SOSP in $\tilde{O}(\epsilon^{-4})$; no explicit d
Fang et al. [45]	SGD with averaging	dispersive noise	ϵ-SOSP in $\tilde{O}(\epsilon^{-3.5})$
Azizian et al. [47]	SGD	large-deviations regime	expected time to global minimizers $\approx \exp(E/\eta)$
Saxe et al. [48]	gradient flow, deep linear	whitened inputs, decoupled init.	exact sigmoidal mode dynamics (16)
Zhang et al. [14]	GD, small init.	additive-unit architectures	saddle-to-saddle along invariant manifolds

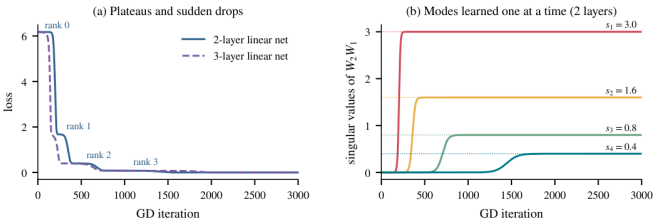


Fig. 6. Original simulation of saddle-to-saddle learning in linear networks trained by GD from small random initialization on a target map with singular values $(3, 1.6, 0.8, 0.4)$. (a) The loss descends in a staircase; each plateau corresponds to a saddle at which the end-to-end map has rank 0, 1, 2, 3. (b) For the two-layer network, the singular values of W_2W_1 are learned one at a time, strongest first, following the sigmoidal law (16) of Saxe et al. [48]. This is the dynamical signature studied in [14].

IV. BEYOND THE STABILITY THRESHOLD (RQ2)

Picture a ball in a narrow valley with a step that is too long for it. Each step overshoots to the opposite wall and lands farther from the floor than before: the multiplier of Section II-F is below -1, and the linear analysis says that the ball flies out. In practice, training with such steps still converges. This section explains how, in order: the stability of SGD and the early phase of training (Section IV-A), catapults (IV-B), the edge of stability (IV-C), the mechanisms that keep it from diverging (IV-D), central flows (IV-E), adaptive and stochastic versions (IV-F), and the effect on where training ends (IV-G).

The results of Section III share an implicit assumption: the step size is small enough that GD approximately follows the gradient flow (Section II-G) and, by the descent lemma (5), decreases the loss monotonically, typically $\eta < 1/\ell$ or $\eta < 2/\ell$ under Assumption 1. Recall from (10) that a minimum with sharpness $\lambda_{\max}(\nabla^2 L) > 2/\eta$ is linearly unstable under GD. Classical analyses therefore choose η small relative to the curvature. Practice does the opposite. This section reviews the literature that confronts that gap, which concerns the right region of Fig. 4. Table IV collects the stability thresholds that appear in it.

A. Precursors: Dynamical Stability and the Break-Even Point

Dynamical stability. Wu et al. [50] asked which global minima can be reached by SGD with a given learning rate and batch size, and studied this question from the perspective of dynamical stability. They introduced a measure of non-uniformity, how much the curvature differs from one training example to another, which, together with the sharpness, characterizes whether a global minimum is stable for a given SGD algorithm, and hence whether that algorithm can reach it. Their analysis shows that the learning rate and the batch size play different roles in this selection. For a loss that is quadratic along the top Hessian eigenvector e_1 (the direction of largest curvature), with sharpness λ_1, Jastrzębski et al. [51] state the resulting condition for SGD to be stable along e_1, citing [50] for its proof, as

$$(1 - \eta\lambda_1)^2 + s^2 \frac{\eta^2(N - S)}{S(N - 1)} \leq 1, \tag{17}$$

where N is the number of training examples, S the batch size, and s^2 the variance over the training examples of the per-example curvature along e_1. Here is where the condition comes from. Along e_1, example i has its own curvature h_i, and the average of the h_i is λ_1. A step on a minibatch B multiplies

the distance ψ along e_1 by the random number $1-\eta\bar{h}_B$, where $\bar{h}_B$ is the average curvature over the batch. SGD is stable in mean square if $\mathbb{E}[\psi^2]$ shrinks. One step multiplies $\mathbb{E}[\psi^2]$ by $\mathbb{E}[(1-\eta\bar{h}_B)^2]$. Using the course identity $\mathbb{E}[X^2] = (\mathbb{E}X)^2 + \mathrm{Var}(X)$, this equals $(1-\eta\lambda_1)^2+\eta^2\mathrm{Var}(\bar{h}_B)$. The variance of the batch average is given by (14), which yields the left side of (17). For full-batch GD ($S = N$) the second term vanishes and (17) reduces to (10). For smaller batches the second term is positive, so the condition is stricter, and SGD can remain stable only where the curvature is smaller or more uniform across examples.

The break-even point. Jastrzębski et al. [51] applied this condition to the early phase of training. In a simplified model in which the sharpness grows until training becomes unstable along e_1, they called the first point at which the left-hand side of (17) reaches one the break-even point. For $S = N$, it is reached at $\lambda_1 = 2/\eta$. They conjectured, and supported with experiments on several classification tasks, that a larger learning rate or a smaller batch size used before this point leads to smaller maximal values of the sharpness and of the spectral norm of the gradient covariance (the strength of the gradient noise in its strongest direction, Section II-H) along the whole trajectory, and to better conditioning of both (Section II-I). They also found that a low learning rate leads to badly conditioned regions even in networks with batch normalization, layers that rescale each unit's activations to zero mean and unit variance over the batch. The early choice of step size therefore shapes the curvature of the regions that training visits later.

B. Catapults

Lewkowycz et al. [13] studied wide networks trained with the squared loss and identified a catapult phase. For learning rates between $2/\lambda_0$, where λ_0 is the sharpness at initialization, and a maximal value $\eta_{\max}$, equal to $4/\lambda_0$ in their solvable model and larger for ReLU networks, the loss first grows sharply. The curvature then decreases until the dynamics become stable, and training converges in a flatter region of the landscape. In their solvable model, the update equations for the output and the curvature are exact, and the loss grows for a number of steps that scales with the logarithm of the width. These dynamics lie outside the lazy description of wide networks. In a very wide network trained with small steps, the weights barely move, so the network behaves like its first-order Taylor expansion in the weights, which is a kernel method of the kind seen in the course (the neural tangent kernel [9]). This regime is called lazy training. Catapults leave it, and in their experiments the best test accuracy was often reached in the catapult phase. A catapult is the simplest instance of the right region of Fig. 4: the iterate is expelled from a region that is too sharp, and the expulsion itself reduces the sharpness.

C. The Edge of Stability

Cohen et al. [12] documented, across architectures and tasks, that full-batch GD typically goes through two phases. In the first, progressive sharpening, the sharpness $\lambda_{\max}(\nabla^2 L)$ rises steadily. In the second, the edge of stability (EoS), the

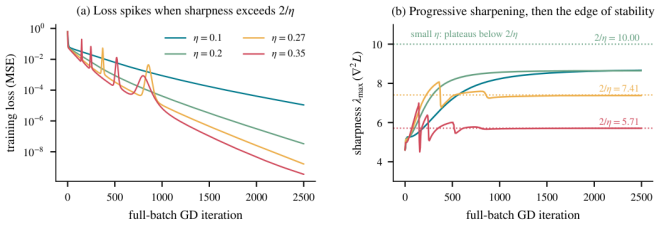


Fig. 7. Original simulation of the edge of stability [12]: a tanh network ($20 \to 96 \to 96 \to 1$) trained by full-batch GD on 256 synthetic regression points, for four step sizes. (a) Training loss. Once the sharpness exceeds $2/\eta$, the loss spikes and then recovers. (b) Sharpness $\lambda_{\max}(\nabla^2 L)$, estimated every 5 steps by power iteration with Hessian-vector products. For small η it saturates below $2/\eta$. For $\eta = 0.27$ and 0.35, it overshoots, is pushed back, and then locks onto $2/\eta$ (dotted lines), the stability threshold of (10).

sharpness reaches approximately $2/\eta$ and then hovers at or slightly above it, while the loss decreases non-monotonically over short time scales but consistently over long ones. With heavy-ball or Nesterov momentum, the same behavior occurs at the correspondingly modified stability threshold (Table IV). GD therefore does not avoid sharp regions by choosing a small step size. It self-tunes the sharpness to the largest value its step size can tolerate.

Fig. 7 reproduces both phases in our own experiment. A two-hidden-layer tanh network is trained with full-batch GD on a synthetic regression task, and the top Hessian eigenvalue is tracked by power iteration with Hessian-vector products (Section II-I). For $\eta \in \{0.1, 0.2\}$, the sharpness saturates near 8.6, below $2/\eta$, and the loss decreases monotonically. For $\eta \in \{0.27, 0.35\}$, progressive sharpening carries the sharpness past $2/\eta$. Each crossing triggers a loss spike followed by a drop in sharpness, and the sharpness then settles at $2/\eta = 7.41$ and 5.71 to within 0.4%, while the loss continues to decrease.

Ahn et al. [52] called this behavior unstable convergence and studied it from first principles. They proved that, under a mild non-degeneracy assumption (roughly: the GD map does not squash sets of positive volume onto sets of zero volume, and $1/\eta$ is not an eigenvalue of the Hessian at any stationary point), GD from almost every initialization cannot converge to a stationary point whose Hessian has either a negative eigenvalue or an eigenvalue above $2/\eta$. The theorem treats strict saddles and overly sharp minima in exactly the same way, as unstable fixed points of the GD map, and its proof follows the stable-manifold argument of Theorem 1. When no flat enough stationary point lies near the trajectory, GD therefore cannot settle anywhere and keeps moving. Ahn et al. identified two causes of unstable convergence, the lack of flat stationary points near the GD trajectory and the existence of forward-invariant sets of the GD dynamics (Section II-G), which keep the unstable iterates from diverging. They characterized the regime by a relative-progress ratio, $\big(L(\theta - \eta\nabla L(\theta)) - L(\theta)\big)/(\eta\|\nabla L(\theta)\|^2)$, the actual change of the loss in one step divided by $\eta\|\nabla L\|^2$. For small steps on a smooth loss it is close to -1, by the first-order Taylor term, and in the stable regime it stays negative. Here it oscillates near zero instead. They also used a directional smoothness, the average curvature of the loss along

the segment that one step travels, which oscillates near $2/\eta$.

D. Mechanisms

Several works explain why GD does not diverge at the EoS.

Phases and an indicator. Li et al. [53] analyzed a two-layer linear network and divided the trajectory into four phases according to whether the sharpness is below or above $2/\eta$ and whether it is rising or falling. They identified the norm of the output-layer weights as an indicator that tracks the sharpness dynamics most of the time.

Self-stabilization. Damian et al. [54] gave a general mechanism based on a third-order Taylor expansion. Let u be the top Hessian eigenvector and $S(\theta) = \lambda_{\max}(\nabla^2 L(\theta))$. When $S > 2/\eta$, the iterate oscillates along u with growing amplitude x, as the multiplier below -1 predicts. Expand the gradient at the displaced point $\theta + xu$ with the third-order Taylor model of Section II-B: $\nabla L(\theta + xu) \approx \nabla L(\theta) + x\nabla^2 L(\theta)u + \frac{x^2}{2}\nabla^3 L(\theta)[u,u]$. Since u is the top eigenvector, $\nabla^2 L(\theta)u = S(\theta)u$, and by Section II-I, $\nabla^3 L(\theta)[u,u] = \nabla S(\theta)$. Hence

$$\nabla L(\theta + xu) \approx \nabla L(\theta) + xS(\theta)u + \tfrac{x^2}{2}\nabla S(\theta), \tag{18}$$

The three terms play different roles. The first is the ordinary gradient. The second flips sign with x and averages out over the oscillation. The third does not flip sign, because $x^2 \geq 0$. A GD step $-\eta\nabla L$ therefore contains the drift $-\eta\frac{x^2}{2}\nabla S$, a step against ∇S, the direction in which the sharpness grows fastest. This drift decreases the sharpness, and more strongly the larger the oscillation. Once S falls below $2/\eta$, the oscillation decays. The net effect, which Damian et al. prove under their assumptions over a long but finite horizon, is that GD at the EoS approximately follows projected gradient descent onto the stable set $\{\theta : S(\theta) \leq 2/\eta\}$: gradient descent that, after each step, is moved back to the nearest point of that set. This accounts for the loss, the sharpness, and the deviation from gradient flow. Fig. 8 sketches the mechanism. The amplification that drives escape from a saddle in Section III is here the same amplification, but with alternating sign, and the third-order term turns it into a restoring force.

Provable entry into the EoS. Arora et al. [55] gave settings in which GD provably operates at the EoS. They studied normalized GD, which can be viewed as GD with the varying step size $\eta/\|\nabla L(\theta_t)\|$, and GD with a constant step size on $\sqrt{L}$ for a smooth loss L with minimum value zero. ($\sqrt{L}$ is not smooth at its minimizers, so its curvature there is effectively unbounded.) Under regularity conditions, in particular a manifold of minimizers, a curve or surface on which the loss is zero, on which the Hessian has constant rank, both provably enter the EoS. The simplest example is their $L(x,y) = (1+x^2)y^2$. Its minimizers form the line $y = 0$, and the sharpness there is $2(1+x^2)$, which is smallest at $x = 0$. Their iterates then oscillate around the manifold of minimizers while following a deterministic flow along it that decreases the sharpness $\lambda_1(\nabla^2 L)$. In the example, both methods bounce across the line $y = 0$ while drifting along it to $x = 0$, which we confirmed numerically, whereas GD with a small step stops at the nearest minimizer. This is an implicit regularization, a preference for certain solutions that comes from the algorithm rather than from a penalty in the loss, and it relies neither on infinitesimal steps nor on gradient noise.

Beyond the edge. Chen and Bruna [56] went beyond the EoS. They analyzed the two-step map $\theta \mapsto G(G(\theta))$, whose fixed points include the period-two orbits of G (Section II-G), and gave local conditions, involving third-order derivatives, under which GD converges to a stable period-two orbit even when single-step stability fails. A quadratic loss can never satisfy them, because its third derivatives are zero: on a quadratic, past $2/\eta$ there is only divergence. They established these conditions in representative learning problems, including teacher-student settings, in which a small student network is trained to imitate a fixed teacher network.

A minimal model. Kalra et al. [57] showed that a two-layer linear "UV" model, $f(x) = v^\top U x$ up to a scaling with two trainable weight arrays U and v, trained on a single example, already exhibits the full sharpness phenomenology. It shows early sharpness reduction, progressive sharpening, and the EoS, and it reveals a period-doubling route to chaos (Section II-G) at the edge of stability as the learning rate increases. Fig. 9 reproduces this bifurcation in our own simulation. As long as $\eta\lambda_\star < 2$, where $\lambda_\star$ is the sharpness of the flattest global minimum, GD from small initialization converges to that minimum. Beyond $\eta\lambda_\star = 2$, no stable minimum exists. The iterates settle into a period-two orbit, period-doubling follows, and a chaotic band appears, while the time-averaged $\eta\lambda_{\max}$ first stays pinned near 2.

E. Central Flows

The mechanisms above analyze the EoS either near a single unstable direction or in small models. Cohen et al. [58] proposed a different strategy. Instead of tracking the oscillatory iterates, they model their time average (Section II-G). Write the GD iterate as $\theta_t = \bar{\theta}_t + x_t u_t$, where $\bar{\theta}_t$ is the time-averaged iterate and x_t the oscillation along the top eigenvector, which averages to zero. The average position moves with the average gradient. Averaging (18) over the oscillation, the middle term $x_t S u$ averages to zero, while the last term averages to $\frac{1}{2}\sigma^2(t)\nabla S$, where $\sigma^2(t)$ is the average of x_t^2, the variance of the oscillation. In continuous time (time counted in steps, Section II-G) this gives the flow

$$\frac{d\theta}{dt} = -\eta\Big[\nabla L(\theta) + \tfrac{1}{2}\sigma^2(t)\nabla S(\theta)\Big], \tag{19}$$

a gradient flow with an implicit sharpness penalty whose strength is the variance $\sigma^2(t)$ of the oscillation. The variance is not a free parameter. By the chain rule, along (19) the sharpness changes at the rate $dS/dt = \langle \nabla S, d\theta/dt\rangle = -\eta\langle \nabla S, \nabla L\rangle - \frac{\eta}{2}\sigma^2\|\nabla S\|^2$. Requiring that the sharpness stay at $2/\eta$, so $dS/dt = 0$, fixes the variance uniquely: $\sigma^2(t) = 2\langle \nabla S, -\nabla L\rangle/\|\nabla S\|^2$. Substituting it back, $d\theta/dt = -\eta\big[\nabla L - \frac{\langle \nabla S, \nabla L\rangle}{\|\nabla S\|^2}\nabla S\big] = -\eta\,\Pi^{\perp}_{\nabla S}\nabla L$, with the projection of Section II-G. The flow is a projected gradient flow that removes exactly the component of the gradient that would increase the sharpness. This nearly recovers the projected dynamics of self-stabilization [54] by a simpler, non-rigorous argument. The general central flow allows any number of Hessian eigenvalues to be at the edge, models the covariance of the

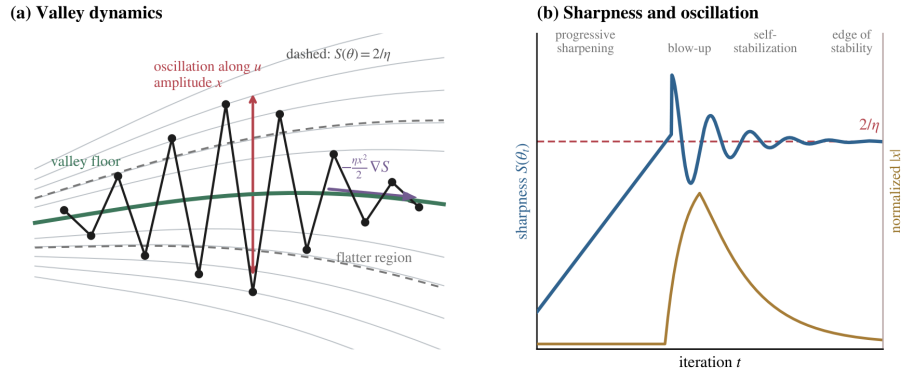


Fig. 8. Self-stabilization [54] (schematic). (a) In a curved valley, the dashed curves mark where the sharpness equals $2/\eta$. When the sharpness exceeds $2/\eta$, the iterates oscillate along the top eigenvector u with growing amplitude x. The third-order term in (18) then induces the drift $-\eta\frac{x^2}{2}\nabla S$, which moves the iterate toward flatter terrain, after which the oscillations decay. (b) The sharpness rises (progressive sharpening), overshoots $2/\eta$, and is pulled back. The oscillation amplitude grows and then decays, and the sharpness settles at $2/\eta$ (edge of stability). GD thereby tracks projected descent onto the set $\{S \le 2/\eta\}$.

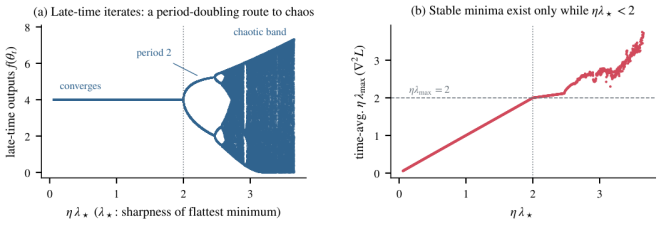


Fig. 9. Original simulation of the route to chaos in a two-layer linear (UV) model, $f = u^\top v/\sqrt{n}$ with $n = 8$, trained by GD on a single example with target 4 from small initialization, for 1400 step sizes. (a) After 4000 burn-in steps, the next 128 outputs are plotted against $\eta\lambda_\star$. The model converges for $\eta\lambda_\star < 2$, oscillates with period two just beyond, and becomes chaotic for larger steps. (b) The time-averaged $\eta\lambda_{\max}(\nabla^2 L)$ equals $\eta\lambda_\star$ while a stable minimum exists, then stays close to the threshold 2 before rising. Compare the phase diagrams of Kalra et al. [57].

oscillations as the solution of a semidefinite complementarity problem, a convex problem whose unknown is a positive semi-definite matrix, as a covariance must be, and reduces to the gradient flow when no eigenvalue is at the edge.

Two properties make the central flow useful. First, its loss decreases monotonically, so the time-averaged loss is a valid potential function, a quantity that decreases at every moment, even though the loss of GD itself is non-monotone. Second, it decreases the loss more slowly than the gradient flow, because the projection removes part of the gradient: the oscillations cost progress through their implicit curvature penalty. Cohen et al. derived analogous flows for scalar and per-coordinate RMSProp, which show how these optimizers implicitly adapt their step sizes to the local curvature and steer towards regions where they can take larger steps. Empirically, the central flows predicted long-term trajectories of GD on a ResNet and a Transformer, and of RMSProp on a Mamba model (three widely used deep architectures), with a high degree of numerical accuracy. The authors stated the limitations explicitly. The derivation is informal, it applies to deterministic training without momentum, and it becomes less accurate for very large learning rates, for some cross-entropy problems, and around large loss spikes.

F. Beyond Full-Batch Gradient Descent

Adaptive methods. Cohen et al. [59] showed that adaptive methods have their own edge of stability. During full-batch training, the largest eigenvalue of the preconditioned Hessian $P^{-1}\nabla^2 L$ (Section II-I) typically equilibrates at the stability threshold of the corresponding update; for Adam with step size η and $\beta_1 = 0.9$ this threshold is $38/\eta$. Similar effects occur during minibatch training, especially as the batch size grows. The behavior differs from that of GD in one important respect. Whereas non-adaptive methods at the EoS are blocked from entering high-curvature regions, adaptive methods at this "adaptive edge of stability" keep advancing into high-curvature regions while adapting the preconditioner to compensate.

Warmup. Kalra and Barkeshli [60] explained learning-rate warmup, starting training with a small step size and raising it gradually to its target, through the same lens. Its main benefit is to let training tolerate a larger target learning rate by steering it towards flatter, better-conditioned regions. As the learning rate rises, loss catapults reduce the sharpness and restore stability. They distinguished a sharpness-reduction regime, in which warmup helps considerably, from a progressive-sharpening regime, in which it helps little. Which regime applies depends on how the network is set up. Standard parameterization and μP are two conventions for how the initialization and the step size of each layer scale with the width, and pre-LayerNorm Transformers normalize the input of every layer. Large standard-parameterization initializations fall in the first regime, and μP or pre-LayerNorm Transformers in the second. For Adam, the relevant quantity is the preconditioned sharpness, which typically starts high. Initializing Adam's second-moment average v (Section II-E) with the squared first gradient instead of zero (GI-Adam) lowers it by a factor of $\sqrt{1-\beta_2}$, which the authors describe as around two orders of magnitude, and acts as an automatic warmup.

Sharpness-aware updates. The same analysis applies to sharpness-aware minimization. For its unnormalized variant

TABLE IV
STABILITY THRESHOLDS OF THE MAIN OPTIMIZERS. IN EACH CASE THE LISTED QUANTITY RISES DURING TRAINING AND THEN STABILIZES NEAR THE THRESHOLD.

Optimizer	Quantity	Threshold	Source
GD	$\lambda_{\max}(\nabla^2 L)$	$2/\eta$	[12]
Heavy ball	$\lambda_{\max}(\nabla^2 L)$	$(2+2\beta)/\eta$	[12]
Nesterov	$\lambda_{\max}(\nabla^2 L)$	$(2+2\beta)/(\eta(1+2\beta))$	[12]
Adam	$\lambda_{\max}(P^{-1}\nabla^2 L)$	$(2+2\beta_1)/((1-\beta_1)\eta)$, i.e. $38/\eta$ for $\beta_1 = 0.9$	[59], [60]
SAM (unnorm.)	$\lambda(1+\rho\lambda)$, $\lambda = \lambda_{\max}$	$2/\eta$	[61]
SGD	batch sharpness (20)	$2/\eta$	[62]
SGD, quadratic model	condition (17)	stricter than $2/\eta$	[50], [51]

(Section II-E), apply the multiplier argument of Section II-F to a quadratic: the gradient at the perturbed point is $H(\theta+\rho H\theta)$, so each eigen-component is multiplied by $1-\eta(\lambda+\rho\lambda^2)$. Agarwala and Dauphin [61] showed accordingly that the stability condition becomes $\eta\lambda(1+\rho\lambda) < 2$ for the curvature eigenvalues λ, where ρ is the perturbation radius. Training then settles at a smaller largest eigenvalue than GD with the same step size, an eigenvalue regularization that arises from the dynamics.

Minibatch SGD. For SGD, the sharpness of the full-batch loss does not lock onto $2/\eta$. Andreyev and Beneventano [62] showed that minibatch SGD trains in a different regime, which they called the edge of stochastic stability. In it, the quantity that stabilizes at $2/\eta$ is the batch sharpness, the expected Rayleigh quotient (Section II-B) of each minibatch Hessian along its own minibatch gradient. Here L_B is the loss averaged over the batch B:

$$\mathrm{BS}(\theta) = \mathbb{E}_B\left[\frac{\nabla L_B(\theta)^\top \nabla^2 L_B(\theta) \nabla L_B(\theta)}{\|\nabla L_B(\theta)\|^2}\right]. \tag{20}$$

Because the full-batch sharpness is generally smaller than the batch sharpness, it is suppressed below $2/\eta$, which fits the long-standing observation that smaller batches and larger step sizes favor flatter minima. The evidence is empirical, for vision classification tasks with the squared loss, and is complemented by a theorem on the local quadratic approximation (2), where crossing $2/\eta$ by any fixed margin forces a catapult. Together with (17), this result suggests that for SGD the relevant notion of stability is stability on the minibatch landscape.

G. Large Steps Change the Destination

Mohtashami et al. [63] argued that the benefits usually attributed to SGD noise are partly benefits of the large step size itself. They showed that these effects persist in noiseless full-batch GD, and they proved that on certain non-convex function classes, GD with a large step size follows a qualitatively different trajectory from small-step GD and can reach a global rather than a local minimum. Galli et al. [64] addressed the question of how large a step size can be when the loss is not globally ℓ-smooth. They gave a unifying definition of "large" step sizes that needs only local Lipschitz or Hölder continuity of the gradient: the Lipschitz condition on small regions only, or the weaker condition $\|\nabla L(a) - \nabla L(b)\| \le K\|a-b\|^p$ with $0 < p \le 1$. They designed adaptive first-order methods, line searches that choose each step size from the observed losses, that provably produce such steps. These methods operate at the EoS from the start of training, with step size times sharpness above 2 throughout. Using them, the sharpness can be driven down to its global minimum. Contrary to expectation, Galli et al. found that reaching globally flat regions, where the sharpness takes its smallest possible value, too early can slow convergence and harm generalization. By exploiting self-stabilization to let GD enter slightly sharper valleys, they turned unsuccessful training runs into successful ones. The slogan "flatter is better" must therefore be qualified.

V. CRITICAL COMPARISON

The previous two sections followed each literature in roughly chronological order. This section steps back and compares them. We ask three questions. What do the two kinds of instability have in common, and where do they differ? What does each result actually guarantee? And which of its assumptions hold for the networks that are trained in practice?

A. One Instability, Two Faces

Ahn et al. [52] made the common structure precise: strict saddles and minima sharper than $2/\eta$ are both unstable fixed points of the GD map, and GD avoids both from almost every initialization. Table V compares the two cases along the questions that the literature asks about them. Three differences stand out.

The question asked. For saddles, the question is how to leave as fast as possible, and the answer is measured in escape time. For the EoS, the question is how to make progress while unstable, and the answer is a description of the time-averaged trajectory. The saddle literature counts iterations; the EoS literature predicts curves.

The role of higher-order terms. At a saddle, the linear instability is the whole story near the critical point, and higher-order terms matter only after escape. At the EoS, the linear instability alone would predict divergence, and it is the third-order term in (18) that keeps training alive. The saddle analysis is local; the EoS analysis is a feedback loop between the iterate and its own curvature.

The role of noise. Noise helps at saddles, because it supplies a component along the direction of escape. Noise does not help stability; it tightens it. Under (17), smaller batches make the curvature threshold stricter, and under (20), SGD locks a minibatch quantity, not the full-batch sharpness, to $2/\eta$. The same randomness that speeds up escape from negative curvature therefore pushes training away from regions of large positive curvature.

B. What Is Guaranteed

Results in both literatures claim success in different senses, and comparing them without care leads to confusion.

TABLE V
TWO FACES OF THE INSTABILITY OF THE GRADIENT-DESCENT MAP.

	Negative curvature (saddles, Section III)	**Excess curvature (EoS, Section IV)**
Condition	$\lambda < 0$ along some eigenvector	$\lambda > 2/\eta$ along some eigenvector
Multiplier $1 - \eta\lambda$	> 1: monotone growth	< -1: growth with alternating sign
What GD does	leaves along the negative-curvature direction; stable set of measure zero [7], [38]	oscillates along the top eigenvector; cannot converge to such points [52]
What limits progress	time near the stable manifold, $\propto \log(1/\delta)$ (11); exponential in d in the worst case [43]	progressive sharpening against oscillation-induced sharpness reduction [54], [58]
Higher-order effect	decides where the escaping trajectory goes	restoring drift $-\eta\frac{x^2}{2}\nabla S$ (18)
Role of noise	supplies a component along the escape direction [6], [44], [45]	makes the threshold stricter (17); batch sharpness locks to $2/\eta$ [62]
Role in learning	saddle-to-saddle simplicity bias [14], [48]	selects flatter regions; catapults [13], [60]; can change the destination [63]
Status of the theory	asymptotic theorems; polynomial-time escape with noise; degenerate saddles open	rigorous in specific settings [54]–[56]; accurate but informal flows [58]

1) *First-order stationarity*, $\|\nabla L\| \le \epsilon$. This is the guarantee of GD and SGD under Assumptions 1 and 3 [27], [30]. An ϵ-FOSP may be a saddle point.
2) *Asymptotic avoidance.* Theorem 1 and its extensions guarantee that the limit, if it exists, is not a strict saddle. They give no rate.
3) *Second-order stationarity*, Definition 4. This is the guarantee of perturbed GD and of the noise-based analyses [6], [8], [44], [45]. It implies approximate local optimality only under the strict-saddle property.
4) *Progress at the EoS.* No classical guarantee applies, because the step size violates the descent lemma. The available results are the monotone decrease of the central-flow loss [58], which is derived informally, and rigorous analyses in specific settings [54]–[56].

A fifth target, implicit in Section IV, is reaching a good minimum, flat enough to be stable at the chosen step size and to generalize. No theorem reviewed here guarantees this for practical networks, although the self-stabilization and catapult analyses [13], [54] explain how the step size biases which minimum is reached.

C. What Is Proved Versus What Is Observed

Table VI lists the main assumptions used in the theory and compares each with what is known about practical training. Three mismatches stand out.

Smoothness and step size. Nearly all saddle-escape results assume a global Lipschitz-gradient constant ℓ and a step size $\eta \le 1/\ell$, which makes GD a descent method. Practical networks violate both. ReLU activations make the gradient discontinuous at their kinks, the points where a unit switches between off and on. Smoothness is better described as growing with the gradient norm [24], which motivates Assumption 4 and its generalizations [25]. Most strikingly, full-batch GD at practical learning rates operates at $\eta\lambda_{\max} \approx 2$ [12], far outside the regime $\eta < 1/\ell$, and hence $\eta\lambda_{\max} < 1$, assumed by Theorem 1. The saddle-avoidance theorems are therefore proved in a regime that practical training leaves early. The convergence of training is then a statement about the joint dynamics of the iterate and the sharpness, which the classical analyses do not capture.

The nature of the noise. Perturbation methods inject isotropic noise (Section II-H) [8], [17], which practitioners never do. Real SGD noise is anisotropic, and its structure is tied to the Hessian. It can be heavy-tailed even in linear regression [46], which challenges the bounded-variance assumption. The CNC and dispersive-noise conditions [44], [45] are closer to practice, since they describe the noise SGD already has. However, they have been checked empirically only on a limited range of models and training stages.

The geometry itself. The strict-saddle property fails for deep linear networks with two or more hidden layers, which possess saddles without negative curvature [37], including the origin, and whenever symmetries or overparameterization create connected sets of degenerate critical points [65]. The property on which the efficient-escape theory rests holds most reliably in the models that are least like deep networks. On the EoS side, the mechanisms assume a well-defined, differentiable sharpness, which needs a gap between the largest and the second-largest Hessian eigenvalue (Section II-I), and the central flow needs the oscillations to be fast relative to the drift of the eigenvectors [58].

These mismatches do not make the theory irrelevant. Rather, they shift the burden of proof. For problems that satisfy the assumptions, the saddle-escape theory is close to complete. For deep networks, both literatures supply mechanisms, such as exponential divergence along negative curvature (11), stuck regions of small volume, and self-stabilization (18), whose relevance must be confirmed empirically.

D. Strengths and Weaknesses of the Two Literatures

The saddle literature is mathematically mature. Its theorems are rigorous and its assumptions are explicit, and for problems that satisfy them, avoidance is almost sure and escape is almost free in iteration count [7], [8]. Its weakness is reach. Its theorems are proved for small step sizes, strict saddles, and, for efficient escape, noise conditions that are rarely checked, and practical training violates all three. The EoS literature has the opposite profile. Its observations are robust across architectures and tasks [12], and its best models predict real training curves with high numerical accuracy [58]. Its weakness is rigor. Its proofs cover simplified losses or small models [55]–[57], its

TABLE VI
STANDARD THEORETICAL ASSUMPTIONS AND THEIR STATUS IN PRACTICAL NEURAL-NETWORK TRAINING.

Assumption	Used by	Status in practice	Evidence
Global ℓ-smoothness, $\eta < 1/\ell$ (Ass. 1)	avoidance [7], most escape bounds	violated: ReLU kinks; smoothness grows with gradient norm; practical GD runs at $\eta\lambda_{\max} \approx 2$	[12], [24]; Fig. 7
Hessian Lipschitz (Ass. 2)	PGD, noise-based escape	plausible for smooth activations; undefined at ReLU kinks	–
Strict-saddle property	efficient escape [6], [8]	proved for matrix and tensor problems; fails for deep linear nets and degenerate valleys	[37], [65]
Isotropic injected noise	PGD, perturbed SGD [17]	not used in practice; SGD noise is anisotropic	[44]
Bounded noise variance (Ass. 3)	SGD analyses	heavy tails can arise even in linear regression	[46]
CNC / dispersive noise	[44], [45]	closer to practice; verified empirically only for limited settings	[44]
Infinitesimal step (gradient flow)	[14], [48]	discrete large steps change the trajectory and the destination	[63], [64]
Full-batch, no momentum	EoS mechanisms, central flows	practical training is stochastic and uses momentum; partial extensions exist	[59], [62]

most predictive tool is derived informally, and its extensions to SGD and Adam are largely empirical [59], [62].

E. Cross-Cutting Lessons

Four lessons emerge from the comparison.

From landscapes to trajectories. Early work asked global questions about the landscape: are all local minima global, and are all saddles strict? For deep networks, the answers are often negative or unknown [37]. Later work asks what the trajectory of a specific algorithm, with a specific step size and initialization, actually visits [14], [48], [54], [58]. Results of the second kind are weaker in scope but much closer to practice.

Step size and noise are mechanisms, not nuisances. Classical theory treats the step size as a parameter to keep small and the noise as an error to control. The large-step literature shows that the step size selects the region, flat or sharp, in which training ends [12], [13], [60], [63], and the noise literature shows that the geometry of SGD noise drives escape [44], [45] and global exploration [47].

Instability is a resource. In both halves of the survey, the instability of the GD map does useful work. Negative curvature carries the iterate from one saddle to the next, implementing a simplicity bias [14]. Excess curvature carries it out of sharp regions, implementing an implicit sharpness penalty [54], [58]. Stability, in the classical sense of $\eta\lambda_{\max} < 2$ and monotone descent, is the exception during training rather than the rule.

Degeneracy is the rule. Symmetries, overparameterization, homogeneous activations, and depth all create degenerate critical points and connected valleys of minima. Strict saddles are a convenient idealization. The more realistic picture, saddle-to-saddle dynamics along invariant manifolds [14], [48] and slow convergence in degenerate valleys [65], suggests that the escape question should be replaced by a question about time spent near degenerate sets.

VI. OPEN PROBLEMS AND FUTURE DIRECTIONS

The comparison of Section V points to specific gaps between what is proved and what is observed. This section turns these gaps into open problems. For each, we summarize the current evidence and suggest a concrete direction.

A. Degenerate and Non-Smooth Saddles

The efficient-escape theory of Section III assumes strict saddles and Lipschitz Hessians. Neural networks violate both. Deep linear networks have saddles without negative curvature [37], overparameterized students converge slowly in degenerate valleys [65] (a student network wider than the single neuron it imitates converges only polynomially fast, instead of exponentially), and ReLU networks are not twice differentiable. At a degenerate saddle, the second-order test of Section II-C is silent, and one must look at third- and fourth-order derivatives. Anandkumar and Ge [66] showed that third-order conditions can be checked efficiently, but that certifying a fourth-order local minimum is NP-hard. A general theory of degenerate saddles therefore cannot exist. The useful question is narrower: do the degenerate saddles that arise from the structure of neural networks, such as the permutation and rescaling symmetries of Section II-A, zero initialization, and neuron duplication (two units with identical weights), admit efficient escape? The results of Cheridito et al. [42] for shallow ReLU networks with scalar input, and the invariant-manifold description of saddle-to-saddle dynamics [14], suggest that structured degeneracy may be tractable where generic degeneracy is not.

B. A Theory of Stochastic and Adaptive Large-Step Training

The optimizers that train modern models, Adam variants with warmup and decay, large learning rates, momentum, and minibatches, are exactly those for which the large-step theory is thinnest. Central flows describe deterministic GD and RMSProp without momentum [58]. The adaptive EoS [59] and the edge of stochastic stability [62] are established mainly empirically. A natural next step is a central flow for SGD, in which the minibatch quantity (20) rather than the full-batch sharpness is held at its threshold, and for Adam, in which the preconditioned sharpness is the state variable [60]. Cohen et al. point out that, because of the randomness, it is not clear whether such a deterministic description exists for stochastic optimizers. The geometry of non-Euclidean optimizers such as Shampoo and Muon [67], [68], which precondition whole weight matrices instead of single coordinates, adds a further

dimension. Such an optimizer takes the steepest step as measured by a different norm, for example the spectral norm of each weight matrix instead of the Euclidean length of the whole parameter vector. Stability thresholds and saddle escape should then be measured in the norm the optimizer uses, not in the Euclidean norm.

C. Does Real SGD Noise Satisfy the Escape Conditions?

Plain SGD provably escapes saddles under CNC [44] or dispersive noise [45]. These conditions are plausible, but they have been verified only on limited models and training stages. Meanwhile, SGD noise can be heavy-tailed [46], and its covariance changes as the sharpness evolves [51]. Systematic measurements across architectures, batch sizes, and training phases, of the noise variance along the bottom Hessian eigenvectors, would settle whether noise-based escape guarantees describe practice. Theory for heavy-tailed noise near saddles is also largely missing.

D. Saddles as Features: Escape Speed Versus Incremental Learning

Saddle-to-saddle dynamics show that the plateaus in which GD lingers can be stages of learning. Each escape adds one unit of complexity, such as a singular value [48] or a neuron [14], [49]. This view conflicts with the goal of efficient escape, which is to leave every saddle as quickly as possible. Is there a trade-off between escape speed and the simplicity bias that incremental learning provides? A precise answer would require coupling escape-time analyses (11) with a measure of the complexity of the function learned at each stage.

E. When the Two Instabilities Meet

The two halves of this survey have been studied separately, yet in practical training they can occur together: training at the EoS oscillates along the top Hessian eigenvectors while, in other directions, the iterate may pass near saddles. The analyses reviewed here treat either one unstable regime or the other. Does the oscillation along high-curvature directions change the escape time along a negative-curvature direction, for example through the third-order coupling in (18)? Does saddle-to-saddle learning from small initialization proceed differently at large step sizes, where the plateaus may be traversed while the sharpness is held at $2/\eta$? A model that contains both a strict saddle and a sharp valley, analyzed with the central-flow methodology, would be a natural first step.

F. Global Exploration and Hitting Times

Almost all results reviewed here are local: they describe escape from one saddle or the behavior near one valley. Azizian et al. [47] took a global view. Using large-deviations theory, they bounded the expected time for SGD to reach the global minimizers, which scales as $\exp(E/\eta)$ in an energy E set by the barriers that SGD may have to climb. Extending such results to the anisotropic, state-dependent noise of minibatch SGD, and to large step sizes at which the iterates oscillate, would show which minima SGD reaches in realistic training times.

VII. Conclusion

This survey has traced how unmodified gradient methods behave when the gradient-descent map is locally unstable. A single multiplier, $1-\eta\lambda$, organizes the story. When $\lambda < 0$, GD leaves strict saddles. It does so from almost every initialization [7], [38], but possibly only after exponential time [43], unless randomness, injected [8] or already present in SGD under suitable conditions [44], [45], places it away from the stable manifold. From small initialization, the sequence of saddles it visits is itself the mechanism of incremental learning [14], [48]. When $\lambda > 2/\eta$, GD cannot settle either [52]. Instead, it oscillates, and the third-order feedback of self-stabilization holds the sharpness at the edge of stability [12], [54], a behavior that central flows now describe with high accuracy for deterministic training [58] and that has adaptive and stochastic counterparts [59], [62].

Across both halves, three patterns recur. Every guarantee is relative to a notion of success, and the links between notions hold only under assumptions that deep networks rarely satisfy exactly. The most predictive analyses are increasingly about trajectories rather than landscapes. And quantities that classical theory treats as obstacles, negative curvature, large step sizes, and gradient noise, turn out to be mechanisms that shape what a network learns. A theory of training that treats instability as part of the algorithm, rather than as a failure to be avoided, appears to be the most promising direction for explaining why simple gradient methods succeed on non-convex losses.

Appendix A
Glossary

Every technical term of the survey, in alphabetical order, with a one-line meaning and the place where it is built. Terms from the background of a first ML course (eigenvalue, gradient, Hessian, expectation, variance, covariance, convexity, singular value decomposition) are used without definition.

Adam, AdaGrad, RMSProp: adaptive methods: a step size per coordinate built from past squared gradients (Sec. II-E).

Almost surely (a.s.): with probability one (Sec. II-G).

Batch sharpness: expected curvature of each minibatch Hessian along its own minibatch gradient; what SGD holds at $2/\eta$ (Sec. IV-F, Eq. (20)).

Bifurcation, period doubling, chaos: qualitative changes of the long-run behavior of GD as η grows: point, 2-cycle, 4-cycle, …, irregular (Sec. II-G).

Break-even point: first point on the trajectory where the SGD stability condition (17) holds with equality (Sec. IV-A).

Catapult: loss spike after which the curvature drops below $2/\eta$ and training settles somewhere flatter (Sec. IV-B).

Center-stable manifold theorem: near an unstable fixed point, the points that stay nearby forever form a lower-dimensional surface (Sec. II-G).

Central flow: differential equation for the time-averaged path of GD at the edge of stability (Sec. IV-E).

CNC: correlated negative curvature: stochastic gradients have a spread along the most negative curvature direction (Sec. III-D).

Condition number κ: $\lambda_{\max}/\lambda_{\min}$ of a positive definite matrix; large κ means a narrow valley (Sec. II-I).

Critical point, FOSP: $\nabla L = 0$; ϵ-FOSP: $\|\nabla L\| \le \epsilon$ (Sec. II-C).

Degenerate saddle: saddle with $\nabla^2 L \succeq 0$: no downhill curvature direction (Sec. II-C).

Descent lemma: $L(\theta - \eta\nabla L) \le L(\theta) - \eta(1 - \ell\eta/2)\|\nabla L\|^2$ (Sec. II-D, Eq. (5)).

Diffeomorphism: smooth map with a smooth inverse; GD is one when $\eta < 1/\ell$ (Sec. II-G).

Dispersive noise: noise that does not concentrate its probability in narrow regions (Sec. III-D).

Edge of stability (EoS): the sharpness rises to $2/\eta$ and hovers there while the loss keeps falling (Sec. IV-C).

Fixed point: $G(\theta) = \theta$; for GD, exactly the critical points (Sec. II-G).

Gradient flow: the small-step limit of GD, $d\theta/ds = -\nabla L$ (Sec. II-G, Eq. (13)).

Heavy ball, Nesterov: momentum methods that reuse a fraction β of the previous step (Sec. II-E).

Heavy tails, tail index α: $P(|X| > t) \sim t^{-\alpha}$; infinite variance if $\alpha \leq 2$ (Sec. II-H).

Hessian-vector product (HVP): $\nabla^2 L\, v$ from two gradients, without forming the Hessian (Sec. II-I).

Invariant set / manifold: a set the dynamics never leave once inside (Sec. II-G).

Isotropic / anisotropic noise: noise covariance proportional to I / not (Sec. II-H).

Large deviations: probabilities of rare events decaying like $e^{-E/\eta}$ (Sec. II-H).

Lazy training, NTK: in very wide networks, weights barely move and the network acts as a kernel method (Sec. IV-B).

Lipschitz, ℓ-smooth: never changes faster than a fixed rate; ℓ-smooth: gradient ℓ-Lipschitz (Sec. II-D).

(L_0, L_1)-smoothness: curvature allowed to grow with the gradient norm (Sec. II-D).

Linear network: network without activations, $f = W_H \cdots W_1 x$ (Sec. II-A).

Measure zero: zero length, area or volume; random starts avoid such sets (Sec. II-G).

Minibatch: random subset of S examples; its gradient is unbiased (Sec. II-H).

Multiplier $1 - \eta\lambda$: factor by which one GD step multiplies the error along an eigenvector (Sec. II-F).

Normalized GD: GD with every step of length η (Sec. II-E).

O, Ω, $\tilde{O}$, poly: growth rates (Sec. II-D).

Oracle model: complexity measured in calls to a gradient oracle (Sec. II-E).

Overparameterized: many more parameters than needed to fit the data (Sec. II-A).

Period-two orbit: GD bouncing forever between two points (Sec. II-G).

Perturbed GD: GD plus a random kick when the gradient is small (Sec. III-C).

Power iteration: repeated $v \leftarrow \nabla^2 L\, v / \|\nabla^2 L\, v\|$; finds the top eigenvector (Sec. II-I).

Preconditioner, preconditioned Hessian: the matrix P of an adaptive step; stability set by $P^{-1}\nabla^2 L$ (Sec. II-E, II-I).

Progressive sharpening: the sharpness rising during training (Sec. IV-C).

Projection $\Pi_v^{\perp}$: $I - vv^\top/\|v\|^2$, removes the component along v (Sec. II-G).

Rayleigh quotient: $u^\top \nabla^2 L\, u$: curvature along a unit vector u (Sec. II-B).

Robbins–Monro conditions: $\sum_t \eta_t = \infty$, $\sum_t \eta_t^2 < \infty$ (Sec. II-E).

Saddle-to-saddle dynamics: training that moves from one saddle to the next, learning one mode at a time (Sec. III-E).

SAM: sharpness-aware minimization: gradient taken at a perturbed point (Sec. II-E).

Self-stabilization: the oscillation lowers the sharpness through the third-order Taylor term (Sec. IV-D).

Sharpness $S(\theta)$: largest Hessian eigenvalue (Sec. II-I).

Simplicity bias: learning simple patterns before complex ones (Sec. III-E).

SOSP: small gradient and no significant negative curvature (Sec. II-C, Def. 4).

Stable set, stable manifold: the starting points from which GD converges to a given fixed point (Sec. II-G).

Strict saddle, strict-saddle property: saddle with $\lambda_{\min} < 0$; every point steep, clearly curved down, or near a good minimum (Sec. II-C).

Strongly convex: $\nabla^2 L \succeq \alpha I$ with $\alpha > 0$ (Sec. II-C).

Unstable convergence: training that keeps progressing while η exceeds $2/\lambda_{\max}$ (Sec. IV-C).

Whitened inputs: inputs transformed to identity covariance (Sec. II-I).

REFERENCES

[1] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, pp. 436–444, 2015.

[2] K. G. Murty and S. N. Kabadi, "Some NP-complete problems in quadratic and nonlinear programming," *Mathematical Programming*, vol. 39, pp. 117–129, 1987.

[3] A. L. Blum and R. L. Rivest, "Training a 3-node neural network is NP-complete," *Neural Networks*, vol. 5, no. 1, pp. 117–127, 1992.

[4] P. Baldi and K. Hornik, "Neural networks and principal component analysis: Learning from examples without local minima," *Neural Networks*, vol. 2, no. 1, pp. 53–58, 1989.

[5] Y. N. Dauphin, R. Pascanu, C. Gulcehre, K. Cho, S. Ganguli, and Y. Bengio, "Identifying and attacking the saddle point problem in high-dimensional non-convex optimization," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2014.

[6] R. Ge, F. Huang, C. Jin, and Y. Yuan, "Escaping from saddle points: Online stochastic gradient for tensor decomposition," in *Conference on Learning Theory (COLT)*, ser. PMLR, vol. 40, 2015.

[7] J. D. Lee, M. Simchowitz, M. I. Jordan, and B. Recht, "Gradient descent only converges to minimizers," in *Conference on Learning Theory (COLT)*, ser. PMLR, vol. 49, 2016, pp. 1246–1257.

[8] C. Jin, R. Ge, P. Netrapalli, S. M. Kakade, and M. I. Jordan, "How to escape saddle points efficiently," in *International Conference on Machine Learning (ICML)*, ser. PMLR, vol. 70, 2017, pp. 1724–1732.

[9] A. Jacot, F. Gabriel, and C. Hongler, "Neural tangent kernel: Convergence and generalization in neural networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

[10] S. Du, J. Lee, H. Li, L. Wang, and X. Zhai, "Gradient descent finds global minima of deep neural networks," in *International Conference on Machine Learning (ICML)*, ser. PMLR, vol. 97, 2019, pp. 1675–1685.

[11] Z. Allen-Zhu, Y. Li, and Z. Song, "A convergence theory for deep learning via over-parameterization," in *International Conference on Machine Learning (ICML)*, 2019.

[12] J. M. Cohen, S. Kaur, Y. Li, J. Z. Kolter, and A. Talwalkar, "Gradient descent on neural networks typically occurs at the edge of stability," in *International Conference on Learning Representations (ICLR)*, 2021.

[13] A. Lewkowycz, Y. Bahri, E. Dyer, J. Sohl-Dickstein, and G. Gur-Ari, "The large learning rate phase of deep learning: The catapult mechanism," 2020, arXiv:2003.02218.

[14] Y. Zhang, A. Saxe, and P. Latham, "Saddle-to-saddle dynamics explains a simplicity bias across neural network architectures," in *International Conference on Learning Representations (ICLR)*, 2026.

[15] L. Bottou, F. E. Curtis, and J. Nocedal, "Optimization methods for large-scale machine learning," *SIAM Review*, vol. 60, no. 2, pp. 223–311, 2018.

[16] P. Jain and P. Kar, "Non-convex optimization for machine learning," *Foundations and Trends in Machine Learning*, vol. 10, no. 3–4, pp. 142–336, 2017.

[17] C. Jin, P. Netrapalli, R. Ge, S. M. Kakade, and M. I. Jordan, "On nonconvex optimization for machine learning: Gradients, stochasticity, and saddle points," *Journal of the ACM*, vol. 68, no. 2, 2021.

[18] R.-Y. Sun, "Optimization for deep learning: An overview," *Journal of the Operations Research Society of China*, vol. 8, pp. 249–294, 2020.

[19] R. Sun, D. Li, S. Liang, T. Ding, and R. Srikant, "The global landscape of neural networks: An overview," *IEEE Signal Processing Magazine*, vol. 37, no. 5, pp. 95–108, 2020.

[20] M. Danilova, P. Dvurechensky, A. Gasnikov, E. Gorbunov, S. Guminov, D. Kamzolov, and I. Shibaev, "Recent theoretical advances in non-convex optimization," in *High-Dimensional Optimization and Probability*. Springer, 2022, pp. 79–163.

[21] S. Hochreiter and J. Schmidhuber, "Flat minima," *Neural Computation*, vol. 9, no. 1, pp. 1–42, 1997.

[22] N. S. Keskar, D. Mudigere, J. Nocedal, M. Smelyanskiy, and P. T. P. Tang, "On large-batch training for deep learning: Generalization gap and sharp minima," in *International Conference on Learning Representations (ICLR)*, 2017.

[23] L. Dinh, R. Pascanu, S. Bengio, and Y. Bengio, "Sharp minima can generalize for deep nets," in *International Conference on Machine Learning (ICML)*, 2017.

[24] J. Zhang, T. He, S. Sra, and A. Jadbabaie, "Why gradient clipping accelerates training: A theoretical justification for adaptivity," in *International Conference on Learning Representations (ICLR)*, 2020.

[25] H. Li, A. Rakhlin, and A. Jadbabaie, "Convergence of Adam under relaxed assumptions," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[26] V. Apidopoulos, I. Lytras, and P. Mertikopoulos, "Taming stochastic gradient descent: Almost sure convergence and saddle-point avoidance under (L_0, L_1)-smoothness," in *International Conference on Machine Learning (ICML)*, ser. PMLR, vol. 306, 2026, pp. 3194–3227.

[27] Y. Nesterov, *Introductory Lectures on Convex Optimization: A Basic Course*. Kluwer Academic Publishers, 2004.

[28] Y. Carmon, J. C. Duchi, O. Hinder, and A. Sidford, "Lower bounds for finding stationary points I," *Mathematical Programming*, vol. 184, pp. 71–120, 2020.

[29] H. Robbins and S. Monro, "A stochastic approximation method," *The Annals of Mathematical Statistics*, vol. 22, no. 3, pp. 400–407, 1951.

[30] S. Ghadimi and G. Lan, "Stochastic first- and zeroth-order methods for nonconvex stochastic programming," *SIAM Journal on Optimization*, vol. 23, no. 4, pp. 2341–2368, 2013.

[31] Y. Arjevani, Y. Carmon, J. C. Duchi, D. J. Foster, N. Srebro, and B. Woodworth, "Lower bounds for non-convex stochastic optimization," *Mathematical Programming*, vol. 199, pp. 165–214, 2023.

[32] B. T. Polyak, "Some methods of speeding up the convergence of iteration methods," *USSR Computational Mathematics and Mathematical Physics*, vol. 4, no. 5, pp. 1–17, 1964.

[33] J. Duchi, E. Hazan, and Y. Singer, "Adaptive subgradient methods for online learning and stochastic optimization," *Journal of Machine Learning Research*, vol. 12, pp. 2121–2159, 2011.

[34] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *International Conference on Learning Representations (ICLR)*, 2015.

[35] Y. Nesterov and B. T. Polyak, "Cubic regularization of Newton method and its global performance," *Mathematical Programming*, vol. 108, pp. 177–205, 2006.

[36] M. Shub, *Global Stability of Dynamical Systems*. Springer, 1987.

[37] K. Kawaguchi, "Deep learning without poor local minima," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2016.

[38] J. D. Lee, I. Panageas, G. Piliouras, M. Simchowitz, M. I. Jordan, and B. Recht, "First-order methods almost always avoid strict saddle points," *Mathematical Programming*, vol. 176, pp. 311–337, 2019.

[39] I. Panageas, G. Piliouras, and X. Wang, "First-order methods almost always avoid saddle points: The case of vanishing step-sizes," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[40] K. Antonakopoulos, P. Mertikopoulos, G. Piliouras, and X. Wang, "AdaGrad avoids saddle points," in *International Conference on Machine Learning (ICML)*, ser. PMLR, vol. 162, 2022.

[41] P. Mertikopoulos, N. Hallak, A. Kavis, and V. Cevher, "On the almost sure convergence of stochastic gradient descent in non-convex problems," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[42] P. Cheridito, A. Jentzen, and F. Rossmannek, "Gradient descent provably escapes saddle points in the training of shallow ReLU networks," *Journal of Optimization Theory and Applications*, 2024.

[43] S. S. Du, C. Jin, J. D. Lee, M. I. Jordan, B. Poczos, and A. Singh, "Gradient descent can take exponential time to escape saddle points," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[44] H. Daneshmand, J. Kohler, A. Lucchi, and T. Hofmann, "Escaping saddles with stochastic gradients," in *International Conference on Machine Learning (ICML)*, 2018.

[45] C. Fang, Z. Lin, and T. Zhang, "Sharp analysis for nonconvex SGD escaping from saddle points," in *Conference on Learning Theory (COLT)*, 2019.

[46] M. Gürbüzbalaban, U. Şimşekli, and L. Zhu, "The heavy-tail phenomenon in SGD," in *International Conference on Machine Learning (ICML)*, 2021.

[47] W. Azizian, F. Iutzeler, J. Malick, and P. Mertikopoulos, "The global convergence time of stochastic gradient descent in non-convex landscapes: Sharp estimates via large deviations," in *International Conference on Machine Learning (ICML)*, 2025.

[48] A. M. Saxe, J. L. McClelland, and S. Ganguli, "Exact solutions to the nonlinear dynamics of learning in deep linear neural networks," in *International Conference on Learning Representations (ICLR)*, 2014.

[49] F. Z. Wu, B. Simsek, and F. Ged, "Loss landscape of shallow ReLU-like neural networks: Stationary points, saddle escape, and network embedding," in *International Conference on Learning Representations (ICLR)*, 2025.

[50] L. Wu, C. Ma, and W. E, "How SGD selects the global minima in over-parameterized learning: A dynamical stability perspective," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 31, 2018, pp. 8279–8288.

[51] S. Jastrzębski, M. Szymczak, S. Fort, D. Arpit, J. Tabor, K. Cho, and K. Geras, "The break-even point on optimization trajectories of deep neural networks," in *International Conference on Learning Representations (ICLR)*, 2020.

[52] K. Ahn, J. Zhang, and S. Sra, "Understanding the unstable convergence of gradient descent," in *International Conference on Machine Learning (ICML)*, ser. PMLR, vol. 162, 2022.

[53] Z. Li, Z. Wang, and J. Li, "Analyzing sharpness along GD trajectory: Progressive sharpening and edge of stability," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[54] A. Damian, E. Nichani, and J. D. Lee, "Self-stabilization: The implicit bias of gradient descent at the edge of stability," in *International Conference on Learning Representations (ICLR)*, 2023.

[55] S. Arora, Z. Li, and A. Panigrahi, "Understanding gradient descent on edge of stability in deep learning," in *International Conference on Machine Learning (ICML)*, ser. PMLR, vol. 162, 2022.

[56] L. Chen and J. Bruna, "Beyond the edge of stability via two-step gradient updates," in *International Conference on Machine Learning (ICML)*, 2023.

[57] D. S. Kalra, T. He, and M. Barkeshli, "Universal sharpness dynamics in neural network training: Fixed point analysis, edge of stability, and route to chaos," in *International Conference on Learning Representations (ICLR)*, 2025.

[58] J. M. Cohen, A. Damian, A. Talwalkar, J. Z. Kolter, and J. D. Lee, "Understanding optimization in deep learning with central flows," in *International Conference on Learning Representations (ICLR)*, 2025.

[59] J. M. Cohen, B. Ghorbani, S. Krishnan, N. Agarwal, S. Medapati, M. Badura, D. Suo, D. Cardoze, Z. Nado, G. E. Dahl, and J. Gilmer, "Adaptive gradient methods at the edge of stability," 2022, arXiv:2207.14484.

[60] D. S. Kalra and M. Barkeshli, "Why warmup the learning rate? Underlying mechanisms and improvements," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[61] A. Agarwala and Y. N. Dauphin, "SAM operates far from home: Eigenvalue regularization as a dynamical phenomenon," in *International Conference on Machine Learning (ICML)*, 2023.

[62] A. Andreyev and P. Beneventano, "Edge of stochastic stability: Revisiting the edge of stability for SGD," 2024, arXiv:2412.20553.

[63] A. Mohtashami, M. Jaggi, and S. U. Stich, "Special properties of gradient descent with large learning rates," in *International Conference on Machine Learning (ICML)*, 2023.

[64] L. Galli, C. Fox, W. Bartolomaeus, M. Schmidt, and H. Rauhut, "Flatland: The adventures of gradient descent with large step sizes," in *International Conference on Machine Learning (ICML)*, ser. PMLR, vol. 306, 2026, pp. 32742–32813.

[65] W. Xu and S. S. Du, "Over-parameterization exponentially slows down gradient descent for learning a single neuron," in *Conference on Learning Theory (COLT)*, 2023.

[66] A. Anandkumar and R. Ge, "Efficient approaches for escaping higher order saddle points in non-convex optimization," in *Conference on Learning Theory (COLT)*, 2016.

[67] V. Gupta, T. Koren, and Y. Singer, "Shampoo: Preconditioned stochastic tensor optimization," in *International Conference on Machine Learning (ICML)*, 2018.

[68] M. Crawshaw, C. Modi, M. Liu, and R. M. Gower, "An exploration of non-Euclidean gradient descent: Muon and its many variants," in *International Conference on Machine Learning (ICML)*, 2026.